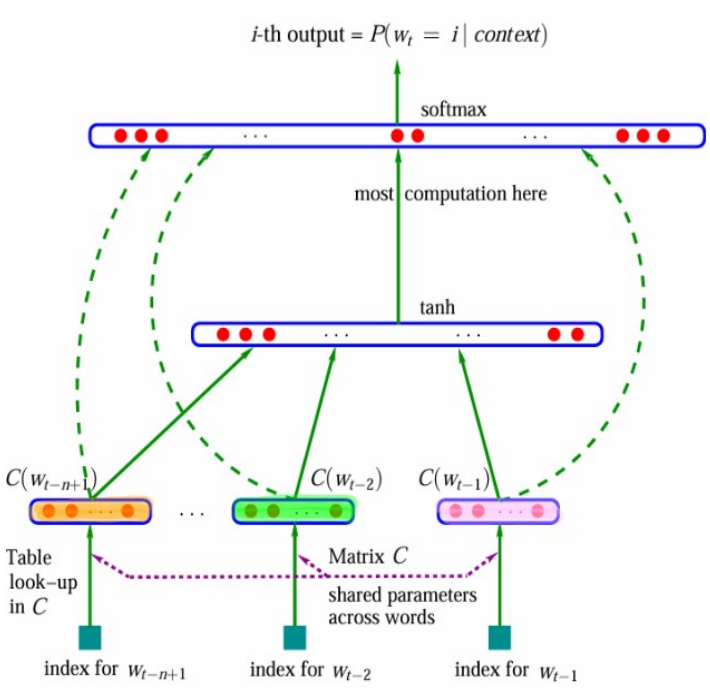


1-1. Neural Probabilistic Language Model (NPLM) - [1]

This video was produced in Korean and translated into English, and the audio was generated by AI (TTS).

www.youtube.com/@meanxai

Word Embedding. All rights reserved by www.youtube.com/@meanxai

MX-AI

1. Neural Probabilistic Language Model (NPLM)

[MXNLP-1-01]
1-1. Yoshua Bengio's 2003 NPLM paper
1-2. Distributed representation of words
1-3. Statistical Language Model (SLM)
1-4. Neural architecture of NPLM

[MXNLP-1-02]
1-5. Implementing a simple NPLM using Keras
- Using Embedding, LSTM, and Cov1D layer in Keras

2. Word2Vec (CBOW, Skip-gram)

[MXNLP-1-03]
2-1. Brief review of the paper
2-2. Model architectures: CBOW, Skip-gram

[MXNLP-1-04]
2-3. Subsampling of frequent words
2-4. Implementation of a simple CBOW model in Keras

[MXNLP-1-05] → 2-5. Implementation of Skip-Gram models in Keras

[MXNLP-1-06] → 2-6. Huffman algorithm and Huffman tree

2-7. Hierarchical softmax
2-8. Implementation of a tiny Skip-gram model using hierarchical softmax in tensorflow
2-9. Negative sampling

Word Embedding. All rights reserved by www.youtube.com/@meanxai

MX-AI

- Yoshua Bengio's 2003 NPLM paper

Journal of Machine Learning Research 3 (2003) 1137–1155 Submitted 4/02; Published 2/03

A Neural Probabilistic Language Model

Yoshua Bengio BENGIOY@IRO.UMONTREAL.CA
Réjean Ducharme DUCHARME@IRO.UMONTREAL.CA
Pascal Vincent VINCENTP@IRO.UMONTREAL.CA
Christian Jauvin JAUVINC@IRO.UMONTREAL.CA
Département d'Informatique et Recherche Opérationnelle
Centre de Recherche Mathématiques
Université de Montréal, Montréal, Québec, Canada

Editors: Jaz Kandola, Thomas Hofmann, Tomaso Poggio and John Shawe-Taylor

Abstract

A goal of statistical language modeling is to learn the joint probability function of sequences of words in a language. This is intrinsically difficult because of the **curse of dimensionality**: a word sequence on which the model will be tested is likely to be different from all the word sequences seen during training. Traditional but very successful approaches based on n-grams obtain generalization by concatenating very short overlapping sequences seen in the training set. We propose to fight the curse of dimensionality by **learning a distributed representation for words** which allows each training sentence to inform the model about an exponential number of semantically neighboring sentences. The model learns simultaneously (1) a distributed representation for each word along with (2) the probability function for word sequences, expressed in terms of these representations. Generalization is obtained because a sequence of words that has never been seen before gets high probability if it is made of words that are similar (in the sense of having a nearby representation) to words forming an already seen sentence. Training such large models (with millions of parameters) within a reasonable time is itself a significant challenge. We report on experiments using neural networks for the probability function, showing on two text corpora that the proposed approach significantly improves on state-of-the-art n-gram models, and that the proposed approach allows to take advantage of longer contexts.

MX-AI

■ Distributed representation of words

[Corpus]

A sentence: **the cat is walking in the bedroom**

The word sequence: **[4, 1, 3, 5, 2, 4, 0]**

[Vocabulary]

0	bedroom
1	cat
2	in
3	is
4	the
5	walking
6	.
7	.
8	.
9	.

[Label Encoding]

the	4
cat	1
is	3
walking	5
in	2
the	4
bedroom	0

[Word sequence]

[One-hot Encoding]

	0 1 2 3 4 5 6 7 8 9
the	0 0 0 0 1 0 0 0 0 0
cat	0 1 0 0 0 0 0 0 0 0
is	0 0 0 1 0 0 0 0 0 0
walking	0 0 0 0 0 1 0 0 0 0
in	0 0 1 0 0 0 0 0 0 0
the	0 0 0 0 1 0 0 0 0 0
bedroom	1 0 0 0 0 0 0 0 0 0

[Local representation]

[Word Embedding]

	0	1	2	3
the	0.42	0.15	-0.52	0.82
cat	0.23	0.25	0.62	-0.41
is	-0.18	0.15	-0.32	0.22
walking	0.71	-0.41	0.12	0.28
in	0.48	0.67	0.53	-0.91
the	0.42	0.15	-0.52	0.82
bedroom	0.18	-0.55	-0.39	-0.37

[Distributed representation]

Word Embedding. All rights reserved by www.youtube.com/@meanxai

MX-AI

■ Statistical language model (SLM)

A statistical language model:

$$P(w_1^T) = \prod_{t=1}^{T} P(w_t | w_1^{t-1}) = P(w_1|w_1^0) \cdot P(w_2|w_1^1) \cdot P(w_3|w_1^2) \cdot P(w_4|w_1^3) \ldots$$

$$w_i^j = (w_i, w_{i+1}, \ldots, w_{j-1}, w_j)$$

A sentence: t**he cat is walking in the bedroom.**

P(The, cat, is, walking, in, the, bedroom) = P(the) x P(cat | the) x P(is | the, cat) x P(walking | the, cat, is) x ...

P(The, cat, is, uum, walking, in, the, bedroom) = ... x P(walking | the, cat, is, uum) ...

n-gram model:

(if n=3: trigrams)

$$P(w_1^T) \simeq \prod_{t=1}^{T} P(w_t | w_{t-n+1}^{t-1}) = P(w_1|w_{-1}^0) \cdot P(w_2|w_0^1) \cdot P(w_3|w_1^2) \cdot P(w_4|w_2^3) \ldots$$

P(The, cat, is, walking, in, the, bedroom) ≒ P(The) x P(cat | The) x P(is | The, cat) x P(walking | cat, is) x P(in | is, walking) ...

Simple example:

Our corpus

The cat is walking in the bedroom.
A dog was running in a room.
The cat is running in a room.
A dog is walking in a bedroom.
The dog was walking in the room.

P(running | dog, was) =

$$\frac{\text{count(dog, was, running)}}{\text{count(dog, was)}} = \frac{1}{2}$$

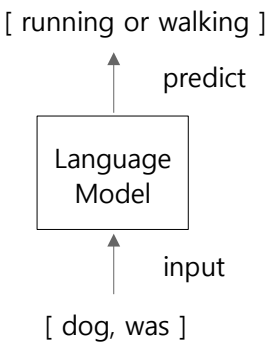
P(barking | dog, was) = $\frac{0}{2}$

P(running | dog, has) = $\frac{0}{0}$

Vocabulary

index	word
0	a
1	bedroom
2	cat
3	dog
4	in
5	is
6	room
7	running
8	the
9	walking
10	was

P(a | dog, was) = 0
P(bedroom | dog, was) = 0
P(cat | dog, was) = 0
P(dog | dog, was) = 0
P(in | dog, was) = 0
P(is | dog, was) = 0
P(room | dog, was) = 0
P(running | dog, was) = 0.5
P(the | dog, was) = 0
P(walking | dog, was) = 0.5
P(was | dog, was) = 0



Word Embedding. All rights reserved by www.youtube.com/@meanxai

■ Neural architecture of NPLM

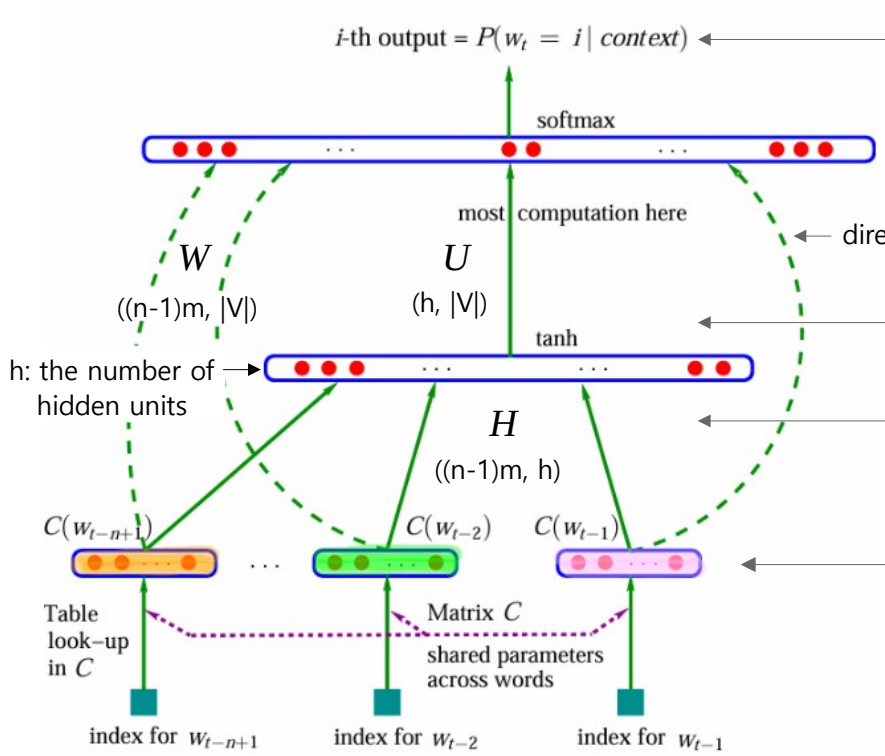
Corpus

The cat is walking in the bedroom.
A dog was running in a room.
The cat is running in a room.
A dog is walking in a bedroom.
The dog was walking in the room.

Vocabulary

index	word
0	a
1	bedroom
2	cat
3	dog
4	in
5	is
6	room
7	running
8	the
9	walking
10	was

Target: walking (9), P(walking | the, cat, is) is highest.



the (8) cat (2) is (5)

(An input example) n = 4 (4-grams)

$$p = softmax(y)$$

$$y = b + U \cdot \tanh(d + H \cdot x) + W \cdot x$$

direct connections (or residual connections)

$$\tanh(d + H \cdot x)$$

x: m m m — $C(w_{t-n+1})$ $C(w_{t-2})$ $C(w_{t-1})$

the concatenation of the word vectors from the matrix C

Matrix C

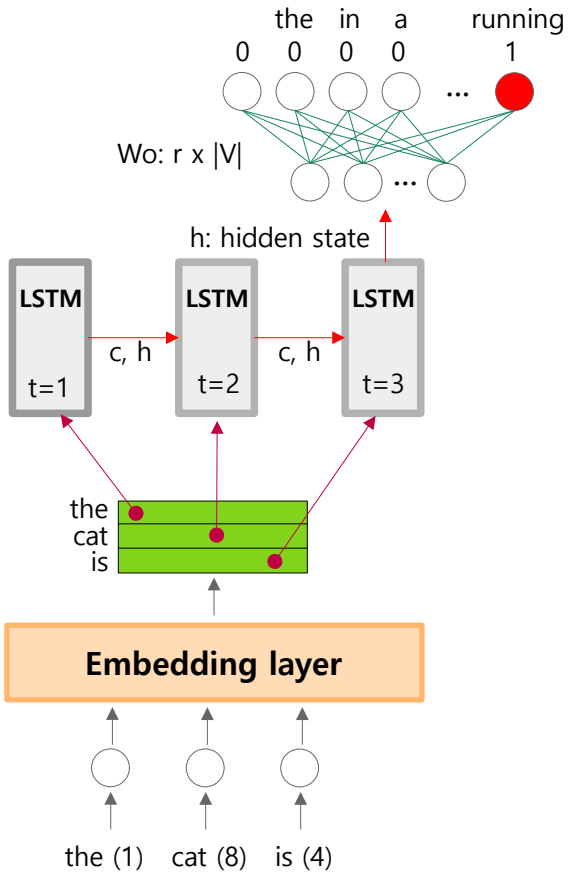
index for w_{t-2} → $C(w_{t-2})$: the word vector for w_{t-2}

index for w_{t-1} → $C(w_{t-1})$

index for w_{t-n+1} → $C(w_{t-n+1})$

$|V| \times m$

|V|: vocabulary size
m: word embedding vector size



1-2. Neural Probabilistic Language Model (NPLM) - [2]

This video was produced in Korean and translated into English, and the audio was generated by AI (TTS).

www.youtube.com/@meanxai

Word Embedding. All rights reserved by www.youtube.com/@meanxai

MX-AI

- Implementing a simple NPLM from scratch using Keras

```python
# [MXNLP-1-01] 1.nplm.py
# A Neural Probabilistic Language Model (NPLM)
# Yoshua Bangio, et. al. 2003, A Neural Probabilistic Language Model
import numpy as np
import tensorflow as tf
from tensorflow.keras.layers import Input, Layer, Dense, Add
from tensorflow.keras.layers import Activation, Flatten
from tensorflow.keras.models import Model
from tensorflow.keras import optimizers
from tensorflow.keras.preprocessing.text import Tokenizer
from nltk.util import ngrams
import matplotlib.pyplot as plt

sentences = ["the cat is walking in the bedroom",
             "a dog was running in a room",
             "the cat is running in a room",
             "a dog is walking in a bedroom",
             "the dog was walking in the room"]

# Data preprocessing
tokenizer = Tokenizer()
tokenizer.fit_on_texts(sentences)
word2idx = tokenizer.word_index
idx2word = {v: k for (k, v) in word2idx.items()}
sequences = tokenizer.texts_to_sequences(sentences) # word sequences

n = 4   # 4-grams
list_ngrams = [list(ngrams(s, n)) for s in sequences]
arry_ngrams = np.array(list_ngrams).reshape(-1, n)

# Training data
x_train = arry_ngrams[:, :-1]
y_train = arry_ngrams[:, -1]
```

word2idx:

```
{'the': 1,
 'in': 2,
 'a': 3,
 'is': 4,
 'walking': 5,
 'dog': 6,
 'room': 7,
 'cat': 8,
 'bedroom': 9,
 'was': 10,
 'running': 11}
```

idx2word:

```
{1: 'the',
 2: 'in',
 3: 'a',
 4: 'is',
 5: 'walking',
 6: 'dog',
 7: 'room',
 8: 'cat',
 9: 'bedroom',
 10: 'was',
 11: 'running'}
```

sequences:

```
[[1, 8, 4, 5, 2, 1, 9],
 [3, 6, 10, 11, 2, 3, 7],
 [1, 8, 4, 11, 2, 3, 7],
 [3, 6, 4, 5, 2, 3, 9],
 [1, 6, 10, 5, 2, 1, 7]]
```

arry_ngrams:

```
array([[ 1,  8,  4,  5],
       [ 8,  4,  5,  2],
       [ 4,  5,  2,  1],
       [ 5,  2,  1,  9],
       [ 3,  6, 10, 11],
       [ 6, 10, 11,  2],
       [10, 11,  2,  3],
       [11,  2,  3,  7],
       [ 1,  8,  4, 11],
       [ 8,  4, 11,  2],
       [ 4, 11,  2,  3],
       [11,  2,  3,  7],
       [ 3,  6,  4,  5],
       [ 6,  4,  5,  2],
       [ 4,  5,  2,  3],
       [ 5,  2,  3,  9],
       [ 1,  6, 10,  5],
       [ 6, 10,  5,  2],
       [10,  5,  2,  1],
       [ 5,  2,  1,  7]])
```

x_train:

```
array([[ 1,  8,  4],
       [ 8,  4,  5],
       [ 4,  5,  2],
       [ 5,  2,  1],
       [ 3,  6, 10],
       [ 6, 10, 11],
       [10, 11,  2],
       [11,  2,  3],
       [ 1,  8,  4],
       [ 8,  4, 11],
       [ 4, 11,  2],
       [11,  2,  3],
       [ 3,  6,  4],
       [ 6,  4,  5],
       [ 4,  5,  2],
       [ 5,  2,  3],
       [ 1,  6, 10],
       [ 6, 10,  5],
       [10,  5,  2],
       [ 5,  2,  1]])
```

y_train:

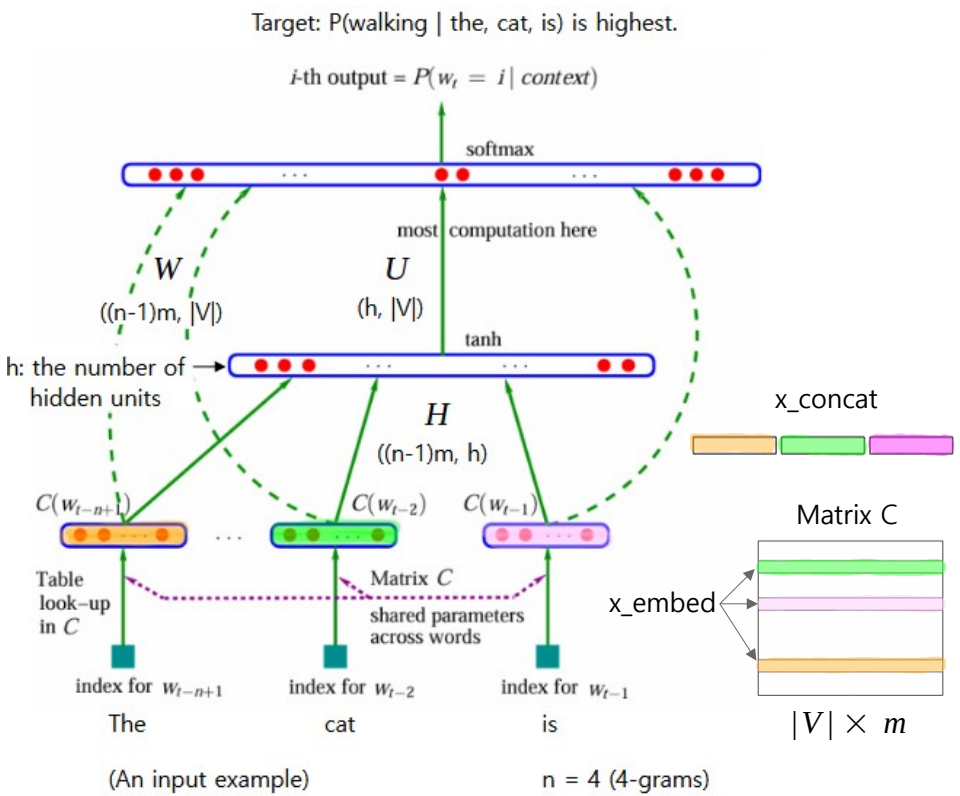
```
array([ 5,
        2,
        1,
        9,
       11,
        2,
        3,
        7,
       11,
        2,
        3,
        7,
        5,
        2,
        3,
        9,
        5,
        2,
        1,
        7])
```

Word Embedding. All rights reserved by www.youtube.com/@meanxai

MX-AI

■ Implementing a simple NPLM from scratch using Keras

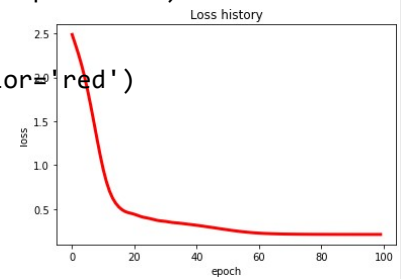
```
m = 8                       # word embedding vector size
V = len(word2idx) + 1       # vocabulary size
h = 64                      # the number of hidden units

# Word Embedding layer
class WordEmbedding(Layer):
    def __init__(self, input_dim, emb_size, name=None):
        super(WordEmbedding, self).__init__(name = name)

        # Matrix C
        C_init = tf.random_normal_initializer()
        self.C = tf.Variable(
            initial_value = C_init(shape=(input_dim, emb_size)),
            trainable = True,
        )

    def call(self, x):
        # Table look-up in C
        selected_rows = tf.gather(self.C, x)  # (None, 3, 8)
        return selected_rows

# Create a NPLM model
x_input = Input(batch_shape = (None, x_train.shape[1]), dtype='int32')
x_embed = WordEmbedding(V, m, name='emb')(x_input) # (None, 3, 8)
x_concat = Flatten()(x_embed)                # concatenate (None, 24)
Hx = Dense(h, activation='tanh')(x_concat)   # tanh(d + Hx)
Wx = Dense(V, use_bias=False)(x_concat)      # Wx (optional)
Ux = Dense(V)(Hx)                            # b + U.tanh(d + Hx)
y = Add()([Wx, Ux])                          # Wx + b + U.tanh(d + Hx)
y_prob = Activation('softmax')(y)
```



MX-AI

■ Implementing a simple NPLM from scratch using Keras

```
model = Model(x_input, y_prob)
model.compile(loss='sparse_categorical_crossentropy',
              optimizer=optimizers.Adam(learning_rate=0.01))
model.summary()

# Fit the data to the model
hist = model.fit(x_train, y_train, epochs=100)

# Loss history
plt.plot(hist.history['loss'], color='red')
plt.title("Loss history")
plt.xlabel("epoch")
plt.ylabel("loss")
plt.show()

# Matrix C
C = model.get_layer('emb').get_weights()[0]

# Word vectors
print("\nWord vectors:")
for word, idx in word2idx.items():
    word_vector = C[idx, :]
    print("{:10s}".format(word), end='')
    print(word_vector.round(3))

# P(w3 | w1, w2, w3)
print("\nP(w4 | w1, w2, w3):")
x = x_train[0]   # "the", "cat", "is"
p = model.predict(x.reshape(1, -1), verbose=0)[0]
print(idx2word[x[0]], idx2word[x[1]], idx2word[x[2]], ':')
for word, idx in word2idx.items():
    print("{:10s}: {:.3f}".format(word, p[idx]))
```




```
# A sentence matrix
print("\nSentence matrix:")
i = 0
print('Sentence:', sentences[i], ':')
print(C[sequences[i]].round(2))
```

```
Word vectors:
the       [-0.293  0.474 -0.133  0.411 -0.274 -0.121  0.239 -0.444]
in        [-0.195 -0.266  0.315  0.242 -0.246 -0.034 -0.299 -0.23 ]
a         [-0.292 -0.186 -0.344 -0.172 -0.184 -0.397  0.278  0.026]
is        [-0.321  0.127 -0.123 -0.406  0.124  0.126 -0.311 -0.268]
walking   [ 0.235 -0.28  -0.409 -0.126 -0.183  0.269 -0.228  0.365]
dog       [ 0.245 -0.487  0.198 -0.458  0.126  0.149  0.329  0.041]
room      [-0.125  0.006  0.046  0.023  0.028 -0.023  0.023  0.045]
cat       [0.221 0.18  0.435 0.347 0.377 0.436 0.234 0.399]
bedroom   [ 0.042  0.029 -0.001  0.017 -0.014  0.001 -0.01  -0.088]
was       [-0.15   0.409 -0.301  0.126  0.476 -0.436 -0.15  -0.281]
running   [ 0.233 -0.1    0.454 -0.435 -0.369 -0.287 -0.321 -0.207]

P(w4 | w1, w2, w3):
the cat is :
the       : 0.000
in        : 0.000
a         : 0.000
is        : 0.000
walking   : 0.501
dog       : 0.000
room      : 0.000
cat       : 0.000
bedroom   : 0.000
was       : 0.000
running   : 0.498

Sentence matrix:
Sentence: the cat is walking in the bedroom :
[[-0.29  0.47 -0.13  0.41 -0.27 -0.12  0.24 -0.44]
 [ 0.22  0.18  0.44  0.35  0.38  0.44  0.23  0.4 ]
 [-0.32  0.13 -0.12 -0.41  0.12  0.13 -0.31 -0.27]
 [ 0.23 -0.28 -0.41 -0.13 -0.18  0.27 -0.23  0.37]
 [-0.19 -0.27  0.32  0.24 -0.25 -0.03 -0.3  -0.23]
 [-0.29  0.47 -0.13  0.41 -0.27 -0.12  0.24 -0.44]
 [ 0.04  0.03 -0.    0.02 -0.01  0.   -0.01 -0.09]]
```

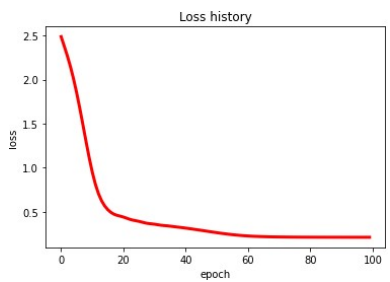
Word Embedding. All rights reserved by www.youtube.com/@meanxai

MX-AI

■ Implementing a simple NPLM using Embedding layer in Keras

```python
# [MXNLP-1-02] 2.nplm(emb).py
# A Neural Probabilistic Language Model (NPLM)
# Yoshua Bangio, et. al. 2003, A Neural Probabilistic Language Model
# Using the Embedding layer in Keras.
import numpy as np
from tensorflow.keras.layers import Input, Dense, Add, Embedding
from tensorflow.keras.layers import Activation, Flatten
from tensorflow.keras.models import Model
from tensorflow.keras import optimizers
from tensorflow.keras.preprocessing.text import Tokenizer
from nltk.util import ngrams
import matplotlib.pyplot as plt

sentences = ["the cat is walking in the bedroom",
             "a dog was running in a room",
             "the cat is running in a room",
             "a dog is walking in a bedroom",
             "the dog was walking in the room"]

# Data preprocessing
tokenizer = Tokenizer()
tokenizer.fit_on_texts(sentences)
word2idx = tokenizer.word_index
idx2word = {v: k for (k, v) in word2idx.items()}
sequences = tokenizer.texts_to_sequences(sentences) # word sequences

n = 4   # 4-grams
list_ngrams = [list(ngrams(s, n)) for s in sequences]
arry_ngrams = np.array(list_ngrams).reshape(-1, n)

# Training data
x_train = arry_ngrams[:, :-1]
y_train = arry_ngrams[:, -1]
```

```python
m = 8                       # word embedding vector size
V = len(word2idx) + 1       # vocabulary size
h = 64                      # the number of hidden units

# Create a NPLM model
x_input = Input(batch_shape = (None, x_train.shape[1]), dtype='int32')
x_embed = Embedding(input_dim=V, output_dim=m, name='emb')(x_input)
x_concat = Flatten()(x_embed)
Hx = Dense(h, activation='tanh')(x_concat)  # tanh(d + Hx)
Wx = Dense(V, use_bias=False)(x_concat)     # Wx (optional)
Ux = Dense(V)(Hx)                           # b + U.tanh(d + Hx)
y = Add()([Wx, Ux])                         # Wx + b + U.tanh(d + Hx)
y_prob = Activation('softmax')(y)

model = Model(x_input, y_prob)
model.compile(loss='sparse_categorical_crossentropy',
              optimizer=optimizers.Adam(learning_rate=0.01))
model.summary()

# Fit the data to the model
hist = model.fit(x_train, y_train, epochs=100)

# Loss history
plt.plot(hist.history['loss'], color='red')
plt.title("Loss history")
plt.xlabel("epoch")
plt.ylabel("loss")
plt.show()

# Matrix C
C = model.get_layer('emb').get_weights()[0]
```



MX-AI

■ Implementing a simple NPLM using Embedding layer in Keras

```
# Word vectors
print("\nWord vectors:")
for word, idx in word2idx.items():
    word_vector = C[idx, :]
    print("{:10s}".format(word), end='')
    print(word_vector.round(3))

# P(w3 | w1, w2, w3)
print("\nP(w4 | w1, w2, w3):")
x = x_train[0]
p = model.predict(x.reshape(1, -1), verbose=0)[0]
print(idx2word[x[0]], idx2word[x[1]], idx2word[x[2]], ':')
for word, idx in word2idx.items():
    print("{:10s}: {:.3f}".format(word, p[idx]))

# A sentence matrix
print("\nSentence matrix:")
i = 0
print('Sentence:', sentences[i], ':')
print(C[sequences[i]].round(2))
```

```
Word vectors:
the       [ 0.55   0.259 -0.337 -0.168 -0.053  0.335  0.341  0.035]
in        [ 0.285  0.287 -0.108  0.174 -0.278  0.246  0.262  0.27 ]
a         [-0.229  0.296 -0.146 -0.22  -0.33   0.129 -0.177  0.399]
is        [-0.278  0.433  0.382 -0.309  0.338 -0.276 -0.056 -0.217]
walking   [ 0.31  -0.338 -0.308  0.28  -0.092  0.276 -0.237 -0.31 ]
dog       [-0.289 -0.583 -0.287  0.316 -0.154 -0.049 -0.287  0.131]
room      [ 0.006 -0.041  0.034 -0.05   0.008 -0.043  0.03   0.019]
cat       [-0.189  0.507 -0.103  0.061 -0.365  0.426 -0.22   0.433]
bedroom   [ 0.021 -0.025 -0.017 -0.009  0.014 -0.05  -0.027 -0.039]
was       [-0.213 -0.092 -0.017 -0.236  0.206 -0.296  0.608 -0.261]
running   [-0.37   0.435 -0.13   0.257 -0.308 -0.07  -0.296  0.377]


P(w4 | w1, w2, w3):
the cat is :
the       : 0.000
in        : 0.000
a         : 0.000
is        : 0.000
walking   : 0.502
dog       : 0.000
room      : 0.000
cat       : 0.000
bedroom   : 0.001
was       : 0.000
running   : 0.498
```

```
Sentence matrix:
Sentence: the cat is walking in the bedroom :
[[ 0.55  0.26 -0.34 -0.17 -0.05  0.34  0.34  0.04]
 [-0.19  0.51 -0.1   0.06 -0.36  0.43 -0.22  0.43]
 [-0.28  0.43  0.38 -0.31  0.34 -0.28 -0.06 -0.22]
 [ 0.31 -0.34 -0.31  0.28 -0.09  0.28 -0.24 -0.31]
 [ 0.29  0.29 -0.11  0.17 -0.28  0.25  0.26  0.27]
 [ 0.55  0.26 -0.34 -0.17 -0.05  0.34  0.34  0.04]
 [ 0.02 -0.02 -0.02 -0.01  0.01 -0.05 -0.03 -0.04]]
```

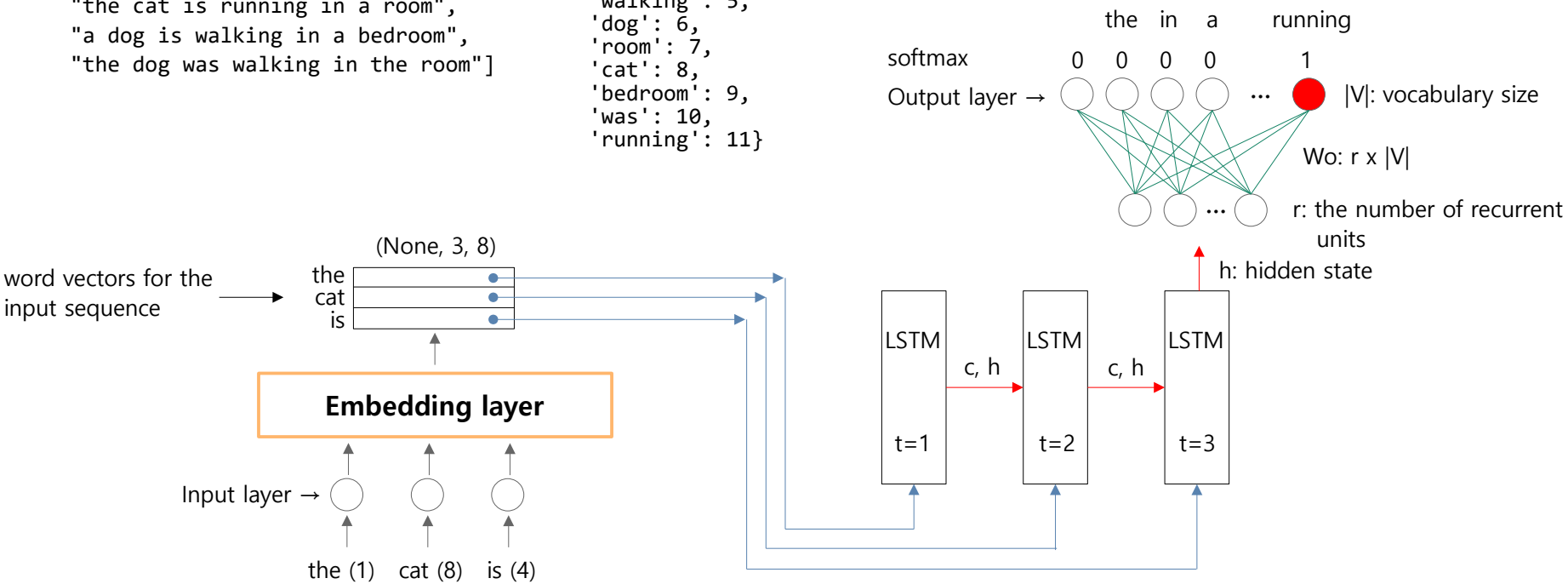
Word Embedding. All rights reserved by www.youtube.com/@meanxai

MX-AI

■ NPLM using a Recurrent Neural Network (RNN)

```
sentences = ["the cat is walking in the bedroom",
             "a dog was running in a room",
             "the cat is running in a room",
             "a dog is walking in a bedroom",
             "the dog was walking in the room"]
```

```
word2idx:
{'the': 1,
 'in': 2,
 'a': 3,
 'is': 4,
 'walking': 5,
 'dog': 6,
 'room': 7,
 'cat': 8,
 'bedroom': 9,
 'was': 10,
 'running': 11}
```



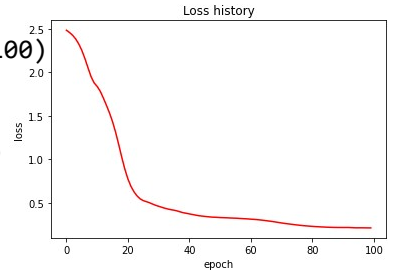
[many-to-one configuration]

MX-AI

■ Implementing a simple NPLM using LSTM network

```python
# [MXNLP-1-02] 3.nplm(lstm).py
# A Neural Probabilistic Language Model (NPLM)
# Yoshua Bangio, et. al. 2003, A Neural Probabilistic Language Model
# Using the Embedding layer and the LSTM layer in Keras.
import numpy as np
from tensorflow.keras.layers import Input, Dense, Embedding
from tensorflow.keras.layers import LSTM
from tensorflow.keras.models import Model
from tensorflow.keras import optimizers
from tensorflow.keras.preprocessing.text import Tokenizer
from nltk.util import ngrams
import matplotlib.pyplot as plt

sentences = ["the cat is walking in the bedroom",
             "a dog was running in a room",
             "the cat is running in a room",
             "a dog is walking in a bedroom",
             "the dog was walking in the room"]

# Data preprocessing
tokenizer = Tokenizer()
tokenizer.fit_on_texts(sentences)
word2idx = tokenizer.word_index
idx2word = {v: k for (k, v) in word2idx.items()}
sequences = tokenizer.texts_to_sequences(sentences) # word sequences

n = 4   # 4-grams
list_ngrams = [list(ngrams(s, n)) for s in sequences]
arry_ngrams = np.array(list_ngrams).reshape(-1, n)

# Training data
x_train = arry_ngrams[:, :-1]
y_train = arry_ngrams[:, -1].reshape(-1, 1)
```

```python
m = 8                     # word embedding vector size
V = len(word2idx) + 1     # vocabulary size
r = 64                    # the number of recurrent units

# Create a NPLM model
x_input = Input(batch_shape = (None, x_train.shape[1]))
x_embed = Embedding(input_dim=V, output_dim=m, name='emb')(x_input)
x_lstm = LSTM(r)(x_embed)
y_output = Dense(V, activation = 'softmax')(x_lstm)

model = Model(x_input, y_output)
model.compile(loss='sparse_categorical_crossentropy',
              optimizer=optimizers.Adam(learning_rate=0.01))
model.summary()

# Fit the data to the model
hist = model.fit(x_train, y_train, epochs=100)

# Loss history
plt.plot(hist.history['loss'], color='red')
plt.title("Loss history")
plt.xlabel("epoch")
plt.ylabel("loss")
plt.show()

# Matrix C
C = model.get_layer('emb').get_weights()[0]
```



MX-AI

■ Implementing a simple NPLM using LSTM network

```
# Word vectors
print("\nWord vectors:")
for word, idx in word2idx.items():
    word_vector = C[idx, :]
    print("{:10s}".format(word), end='')
    print(word_vector.round(3))

# P(w3 | w1, w2, w3)
print("\nP(w4 | w1, w2, w3):")
x = x_train[0]
p = model.predict(x.reshape(1, -1), verbose=0)[0]
print(idx2word[x[0]], idx2word[x[1]], idx2word[x[2]], ':')
for word, idx in word2idx.items():
    print("{:10s}: {:.3f}".format(word, p[idx]))

# A sentence matrix
print("\nSentence matrix:")
i = 0
print('Sentence:', sentences[i], ':')
print(C[sequences[i]].round(2))
```

```
Word vectors:
the       [-0.529  0.405 -0.25   0.418 -0.437  0.46  -0.655 -0.224]
in        [-0.303  0.237  0.2    0.047 -0.291  0.32  -0.374 -0.307]
a         [-0.136  0.4   -0.208  0.291 -0.033  0.144 -0.064  0.392]
is        [ 0.499 -0.435  0.086 -0.526  0.553 -0.384  0.468 -0.163]
walking   [ 0.323  0.226  0.151 -0.158 -0.047  0.474  0.252  0.118]
dog       [ 0.016  0.198 -0.311  0.238  0.341 -0.076  0.232  0.141]
room      [ 0.003  0.017  0.045  0.018  0.016 -0.    -0.019 -0.006]
cat       [ 0.521  0.396 -0.328  0.099  0.482 -0.242  0.545  0.357]
bedroom   [-0.049 -0.047 -0.03   0.048  0.037 -0.043 -0.009  0.042]
was       [ 0.032 -0.033  0.149 -0.141 -0.081 -0.107  0.152  0.165]
running   [ 0.273 -0.511  0.505 -0.491  0.415  0.172 -0.292 -0.552]


P(w4 | w1, w2, w3):
the cat is :
the       : 0.000
in        : 0.000
a         : 0.000
is        : 0.000
walking   : 0.524
dog       : 0.000
room      : 0.000
cat       : 0.000
bedroom   : 0.000
was       : 0.000
running   : 0.476
```

```
Sentence matrix:
Sentence: the cat is walking in the bedroom :
[[-0.53  0.41 -0.25  0.42 -0.44  0.46 -0.66 -0.22]
 [ 0.52  0.4  -0.33  0.1   0.48 -0.24  0.55  0.36]
 [ 0.5  -0.43  0.09 -0.53  0.55 -0.38  0.47 -0.16]
 [ 0.32  0.23  0.15 -0.16 -0.05  0.47  0.25  0.12]
 [-0.3   0.24  0.2   0.05 -0.29  0.32 -0.37 -0.31]
 [-0.53  0.41 -0.25  0.42 -0.44  0.46 -0.66 -0.22]
 [-0.05 -0.05 -0.03  0.05  0.04 -0.04 -0.01  0.04]]
```

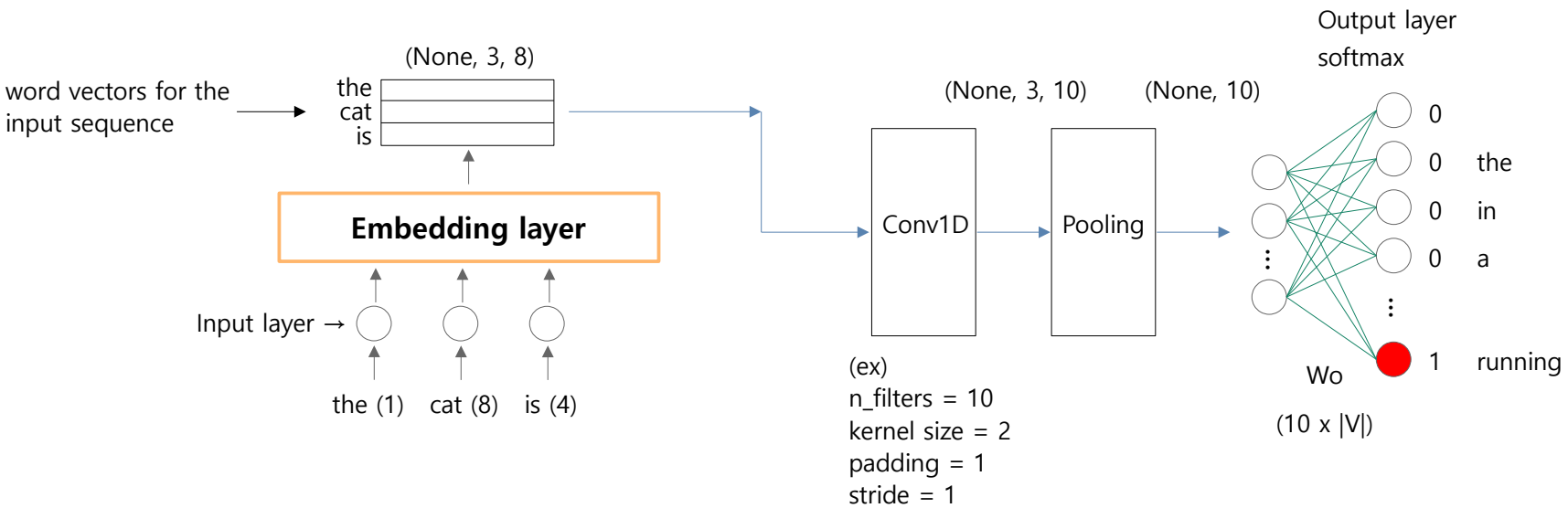
Word Embedding. All rights reserved by www.youtube.com/@meanxai

MX-AI

■ NPLM using a Convolutional Neural Network (CNN)

```
sentences = ["the cat is walking in the bedroom",
             "a dog was running in a room",
             "the cat is running in a room",
             "a dog is walking in a bedroom",
             "the dog was walking in the room"]
```

```
word2idx:
{'the': 1,
 'in': 2,
 'a': 3,
 'is': 4,
 'walking': 5,
 'dog': 6,
 'room': 7,
 'cat': 8,
 'bedroom': 9,
 'was': 10,
 'running': 11}
```

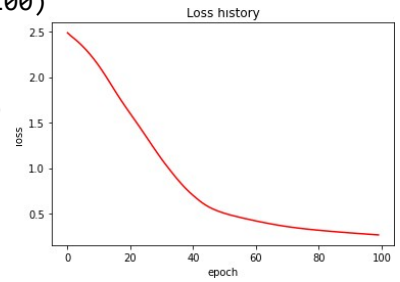


MX-AI

■ Implementing a simple NPLM using a 1D Convolutional Neural Network

```python
# [MXNLP-1-02] 4.nplm(cnn).py
# A Neural Probabilistic Language Model (NPLM)
# Yoshua Bangio, et. al. 2003, A Neural Probabilistic Language Model
# Using the Embedding layer and the Conv1D layer in Keras.
import numpy as np
from tensorflow.keras.layers import Input, Dense, Embedding
from tensorflow.keras.layers import Conv1D, GlobalMaxPooling1D
from tensorflow.keras.models import Model
from tensorflow.keras import optimizers
from tensorflow.keras.preprocessing.text import Tokenizer
from nltk.util import ngrams
import matplotlib.pyplot as plt

sentences = ["the cat is walking in the bedroom",
             "a dog was running in a room",
             "the cat is running in a room",
             "a dog is walking in a bedroom",
             "the dog was walking in the room"]

# Data preprocessing
tokenizer = Tokenizer()
tokenizer.fit_on_texts(sentences)
word2idx = tokenizer.word_index
idx2word = {v: k for (k, v) in word2idx.items()}
sequences = tokenizer.texts_to_sequences(sentences) # word sequences

n = 4   # 4-grams
list_ngrams = [list(ngrams(s, n)) for s in sequences]
arry_ngrams = np.array(list_ngrams).reshape(-1, n)

# Training data
x_train = arry_ngrams[:, :-1]
y_train = arry_ngrams[:, -1].reshape(-1, 1)
```

```python
m = 8                       # word embedding vector size
V = len(word2idx) + 1       # vocabulary size
n_filters = 10              # the number of filters in Conv1D
filter_size = 2             # the size of the convolution window

# Create a NPLM model
x_input = Input(batch_shape = (None, x_train.shape[1]))
x_embed = Embedding(input_dim=V, output_dim=m, name='emb')(x_input)

x_conv = Conv1D(n_filters, filter_size, padding='same')(x_embed)
x_pool = GlobalMaxPooling1D()(x_conv)
y_output = Dense(V, activation = 'softmax')(x_pool)

model = Model(x_input, y_output)
model.compile(loss='sparse_categorical_crossentropy',
              optimizer=optimizers.Adam(learning_rate=0.01))
model.summary()

# Fit the data to the model
hist = model.fit(x_train, y_train, epochs=100)

# Loss history
plt.plot(hist.history['loss'], color='red')
plt.title("Loss history")
plt.xlabel("epoch")
plt.ylabel("loss")
plt.show()
```



MX-AI

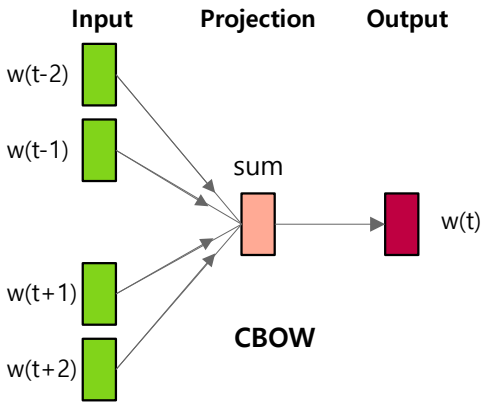
■ Implementing a simple NPLM using a 1D Convolutional Neural Network

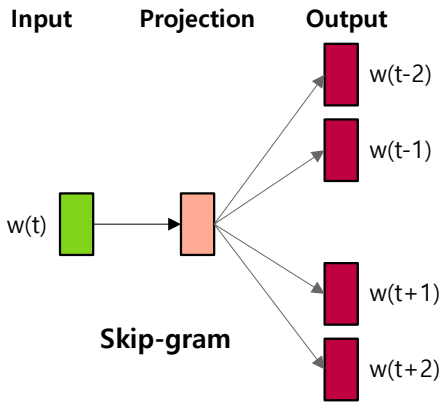
```python
# Matrix C
C = model.get_layer('emb').get_weights()[0]

# Word vectors
print("\nWord vectors:")
for word, idx in word2idx.items():
    word_vector = C[idx, :]
    print("{:10s}".format(word), end='')
    print(word_vector.round(3))

# P(w3 | w1, w2, w3)
print("\nP(w4 | w1, w2, w3):")
x = x_train[0]
p = model.predict(x.reshape(1, -1), verbose=0)[0]
print(idx2word[x[0]], idx2word[x[1]], idx2word[x[2]], ':')
for word, idx in word2idx.items():
    print("{:10s}: {:.3f}".format(word, p[idx]))

# A sentence matrix
print("\nSentence matrix:")
i = 0
print('Sentence:', sentences[i], ':')
print(C[sequences[i]].round(2))
```

```
Word vectors:
the       [-0.434 -0.471  0.45  -0.369  0.49   0.396  0.597 -0.201]
in        [-0.626 -0.502  0.222 -0.606  0.301  0.078 -0.706  0.008]
a         [-0.397 -0.763  0.575 -0.066  0.53   0.57  -0.037  0.831]
is        [ 0.478 -0.     0.029 -0.434 -0.112  0.329  0.501 -0.509]
walking   [-0.764  0.53   0.673 -0.042  0.519  0.262 -0.199 -0.769]
dog       [ 0.439  0.562 -0.605  0.564 -0.622 -0.601  0.217  0.5  ]
room      [ 0.015  0.032 -0.001 -0.012  0.029  0.001 -0.006  0.019]
cat       [ 0.638  0.663 -0.588  0.521 -0.472 -0.627 -0.675  0.53 ]
bedroom   [-0.002  0.013  0.02  -0.014  0.04  -0.011 -0.014  0.041]
was       [ 0.241 -0.176 -0.239 -0.082 -0.318 -0.172  0.596  0.29 ]
running   [ 0.433  0.236  0.283 -0.586  0.527  0.77  -0.783  0.006]


P(w4 | w1, w2, w3):
the cat is :
the       : 0.000
in        : 0.007
a         : 0.000
is        : 0.000
walking   : 0.516
dog       : 0.000
room      : 0.000
cat       : 0.000
bedroom   : 0.000
was       : 0.000
running   : 0.477
```

```
Sentence matrix:
Sentence: the cat is walking in the bedroom :
[[-0.43 -0.47  0.45 -0.37  0.49  0.4   0.6  -0.2 ]
 [ 0.64  0.66 -0.59  0.52 -0.47 -0.63 -0.68  0.53]
 [ 0.48 -0.    0.03 -0.43 -0.11  0.33  0.5  -0.51]
 [-0.76  0.53  0.67 -0.04  0.52  0.26 -0.2  -0.77]
 [-0.63 -0.5   0.22 -0.61  0.3   0.08 -0.71  0.01]
 [-0.43 -0.47  0.45 -0.37  0.49  0.4   0.6  -0.2 ]
 [-0.    0.01  0.02 -0.01  0.04 -0.01 -0.01  0.04]]
```

Word Embedding. All rights reserved by www.youtube.com/@meanxai





1-3. Word2Vec - [1]
CBOW and Skip-gram

This video was produced in Korean and translated into English, and the audio was generated by AI (TTS).

www.youtube.com/@meanxai

Word Embedding. All rights reserved by www.youtube.com/@meanxai

MX-AI

- Word2Vec papers

Efficient Estimation of Word Representations in Vector Space

Tomas Mikolov
Google Inc., Mountain View, CA
tmikolov@google.com

Kai Chen
Google Inc., Mountain View, CA
kaichen@google.com

Greg Corrado
Google Inc., Mountain View, CA
gcorrado@google.com

Jeffrey Dean
Google Inc., Mountain View, CA
jeff@google.com

Abstract

We propose two novel model architectures for computing
representations of words from very large data sets. The c
representations is measured in a word similarity task, and
compared to the previously best performing techniques b
types of neural networks. We observe large improvement
much lower computational cost, i.e. it takes less than a d
quality word vectors from a 1.6 billion words data set. Fu
show that these vectors provide state-of-the-art performance on our test set for measuring syntactic and semantic word similarities.

key points

- Distributed representation based on NNLM
- Simple model on huge data set
- Huffman tree and hierarchical softmax
- Parallel Training of Neural Networks
- Multiple degrees of similarity
- Subsampling of the frequent words
- Negative sampling

Distributed Representations of Words and Phrases and their Compositionality

Tomas Mikolov
Google Inc.
Mountain View
mikolov@google.com

Ilya Sutskever
Google Inc.
Mountain View
ilyasu@google.com

Kai Chen
Google Inc.
Mountain View
kai@google.com

Greg Corrado
Google Inc.
Mountain View
gcorrado@google.com

Jeffrey Dean
Google Inc.
Mountain View
jeff@google.com

Abstract

ntinuous Skip-gram model is an efficient method for
buted vector representations that capture a large number
mantic word relationships. In this paper we present
prove both the quality of the vectors and the training
the frequent words we obtain significant speedup and
rd representations. We also describe a simple alternative
called negative sampling.
ord representations is their indifference to word order
and their inability to represent idiomatic phrases. For example, the meanings of "Canada" and "Air" cannot be easily combined to obtain "Air Canada". Motivated by this example, we present a simple method for finding phrases in text, and show that learning good vector representations for millions of phrases is possible.

MX-AI

- Distributed representation based on NNLM
- Simple model on huge data set

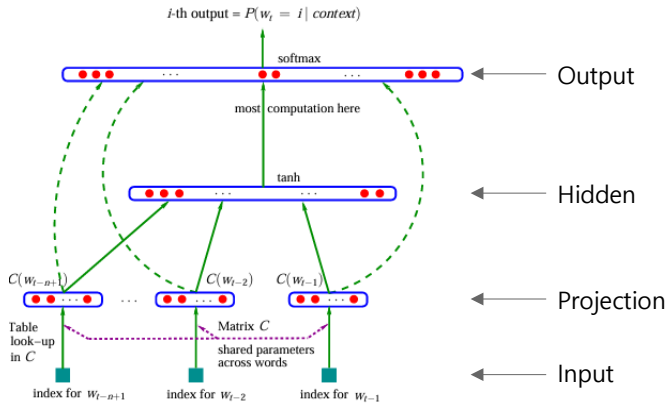
[Local representation]

	0	1	2	3	4	5	6	7	8	9
the	0	0	0	0	1	0	0	0	0	0
cat	0	1	0	0	0	0	0	0	0	0
is	0	0	0	1	0	0	0	0	0	0
walking	0	0	0	0	0	1	0	0	0	0
bedroom	1	0	0	0	0	0	0	0	0	0

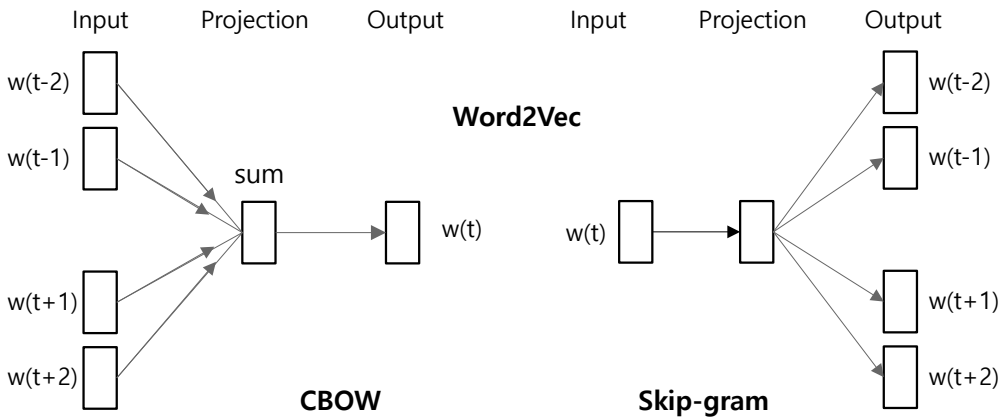
[Distributed representation]

	0	1	2	3
the	0.42	0.15	-0.52	0.82
cat	0.23	0.25	0.62	-0.41
is	-0.18	0.15	-0.32	0.22
walking	0.71	-0.41	0.12	0.28
bedroom	0.18	-0.55	-0.39	-0.37

NPLM



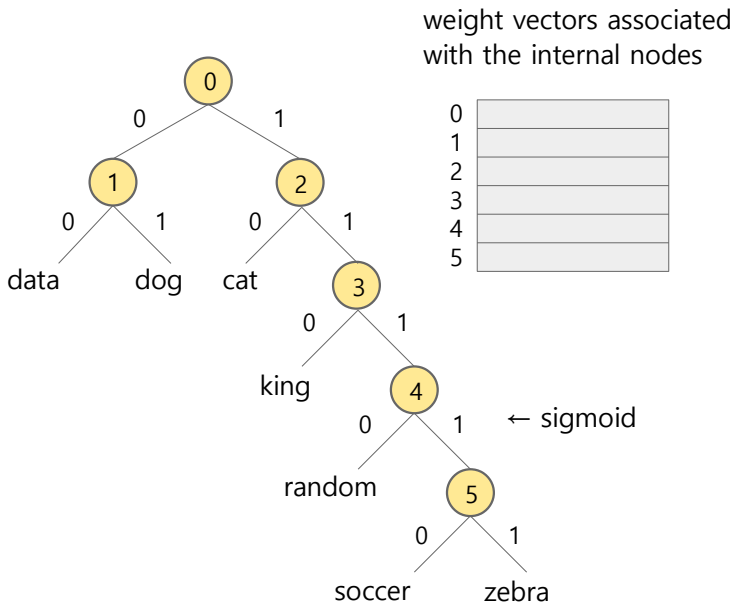
Word2Vec



Word Embedding. All rights reserved by www.youtube.com/@meanxai

MX-AI

- Huffman tree and hierarchical softmax



	standard softmax	hierachical softmax
complexity:	**O(V)**	**O(log(V))**

- Parallel Training of Neural Networks

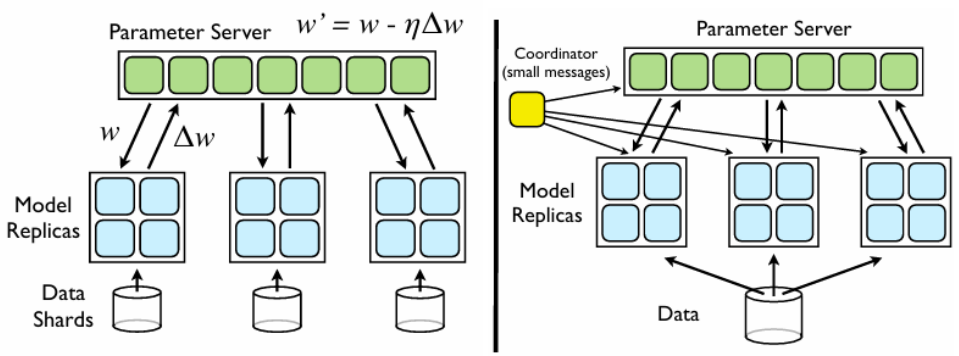


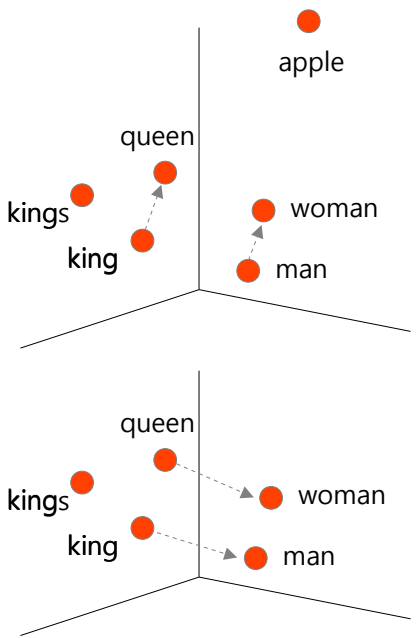
Figure 2: Left: Downpour SGD. Model replicas asynchronously fetch parameters w and push gradients Δw to the parameter server. Right: Sandblaster L-BFGS. A single 'coordinator' sends small messages to replicas and the parameter server to orchestrate batch optimization.

[Source: Jeffrey Dean, et, al., 2012, Large Scale Distributed Deep Networks]

Word Embedding. All rights reserved by www.youtube.com/@meanxai

MX-AI

- Multiple degrees of similarity



[word embedding vector space]

vector("King") - vector("Man") + vector("Woman") ≒ vector("Queen")

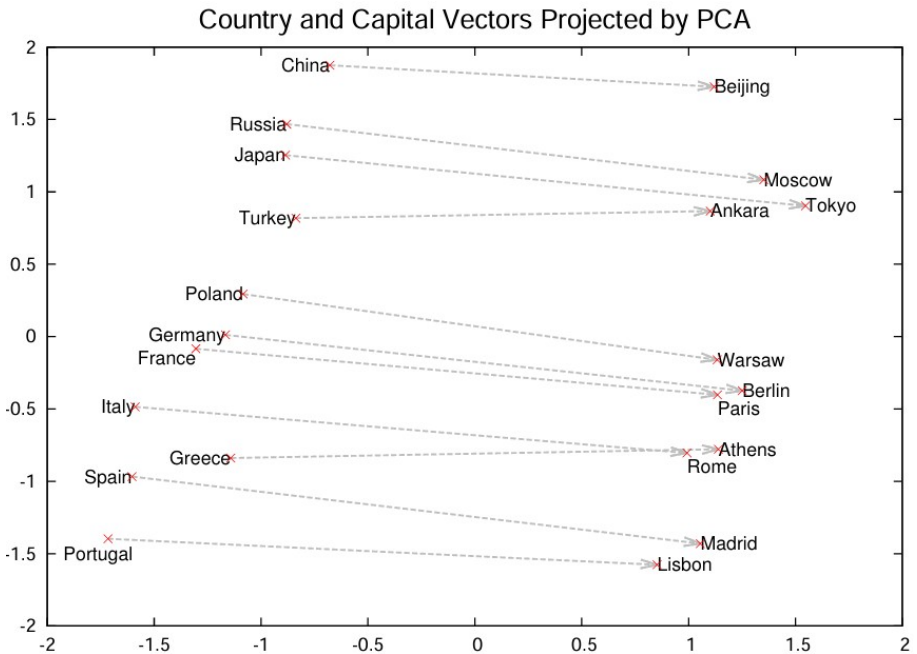


Figure 2: Two-dimensional PCA projection of the 1000-dimensional Skip-gram vectors of countries and their capital cities. The figure illustrates ability of the model to automatically organize concepts and learn implicitly the relationships between them, as during the training we did not provide any supervised information about what a capital city means.

[Source: Tomas Mikolov, 2013, Distributed Representations of Words and Phrases and their Compositionality]

MX-AI

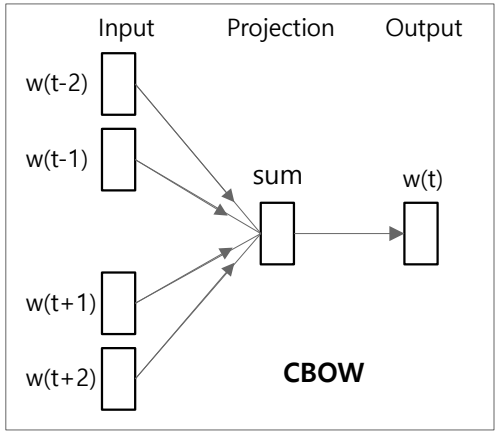
■ Continuous Bag-of-Words (CBOW) Model

Corpus:

```
the cat is walking in the bedroom
a dog was running in a room
the cat is running in a room
a dog is walking in a bedroom
the dog was walking in the room
```

Vocabulary:

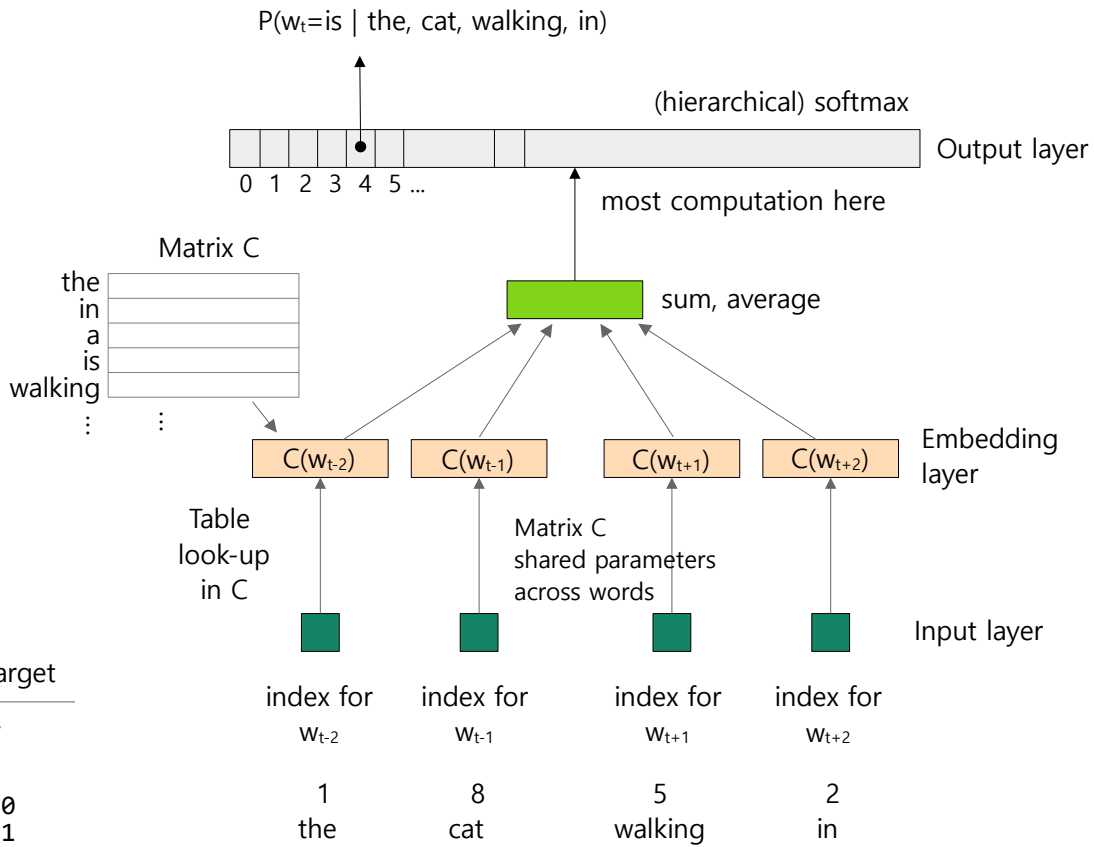
```
{'the': 1,
 'in': 2,
 'a': 3,
 'is': 4,
 'walking': 5,
 'dog': 6,
 'room': 7,
 'cat': 8,
 'bedroom': 9,
 'was': 10,
 'running': 11}
```



Training data:

w_{t-2}	w_{t-1}	w_{t+1}	w_{t+2}	w_t
the	cat	walking	in	is
cat	is	in	the	walking
is	walking	the	bedroom	in
a	dog	running	in	was
dog	was	in	a	running
⋮				

(context) input				target
1	8	5	2	4
8	4	2	1	5
4	5	1	9	2
3	6	11	2	10
6	10	2	3	11
⋮				⋮



Word Embedding. All rights reserved by www.youtube.com/@meanxai

MX-AI

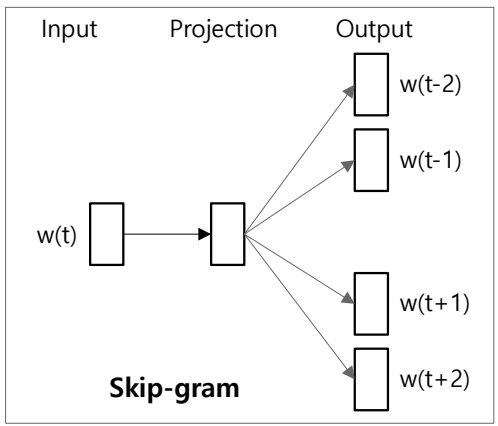
■ Continuous Skip-gram Model

Corpus:

```
the cat is walking in the bedroom
a dog was running in a room
the cat is running in a room
a dog is walking in a bedroom
the dog was walking in the room
```

Vocabulary:

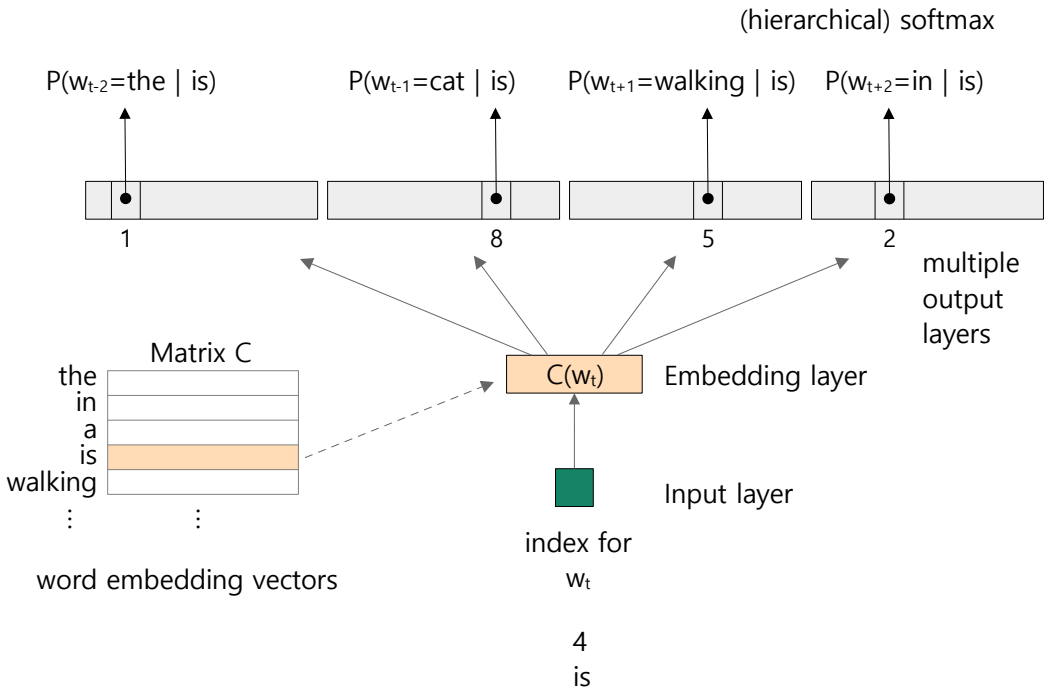
```
{'the': 1,
 'in': 2,
 'a': 3,
 'is': 4,
 'walking': 5,
 'dog': 6,
 'room': 7,
 'cat': 8,
 'bedroom': 9,
 'was': 10,
 'running': 11}
```



Training data:

w_t	w_{t-2}	w_{t-1}	w_{t+1}	w_{t+2}
is	the	cat	walking	in
walking	cat	is	in	the
in	is	walking	the	bedroom
was	a	dog	running	in
running	dog	was	in	a
⋮				

(target) input	(context) output			
4	1	8	5	2
5	8	4	2	1
2	4	5	1	9
10	3	6	11	2
11	6	10	2	3
	⋮			⋮



MX-AI

■ Continuous Skip-gram Model

Corpus:

```
the cat is walking in the bedroom
a dog was running in a room
the cat is running in a room
a dog is walking in a bedroom
the dog was walking in the room
```

Vocabulary:

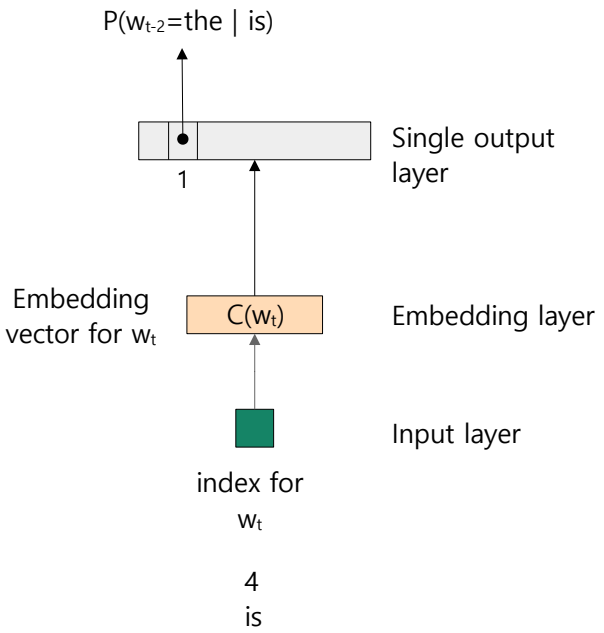
```
{'the': 1,
 'in': 2,
 'a': 3,
 'is': 4,
 'walking': 5,
 'dog': 6,
 'room': 7,
 'cat': 8,
 'bedroom': 9,
 'was': 10,
 'running': 11}
```

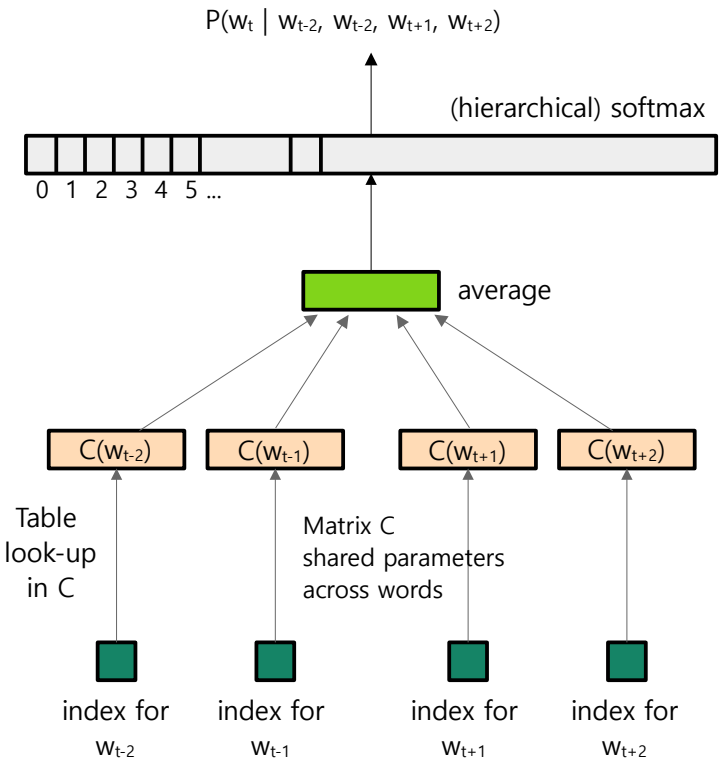
Training data:

w_t	w_{t-2}	w_{t-1}	w_{t+1}	w_{t+2}
is	the	cat	walking	in
walking	cat	is	in	the
in	is	walking	the	bedroom
was	a	dog	running	in
running	dog	was	in	a
⋮				

(target) input	(context) output			
4	1	8	5	2
5	8	4	2	1
2	4	5	1	9
10	3	6	11	2
11	6	10	2	3
	⋮			⋮

(target) input	(context) output
4	1
4	8
4	5
4	2
5	8
5	4
5	2
5	1
2	4
2	5
2	1
2	9
10	3
10	6
10	11
10	2
11	6
11	10
11	2
11	3
⋮	⋮





1-4. Word2Vec - [2]

Implementing a simple CBOW model

This video was produced in Korean and translated into English, and the voice was generated using TTS.

www.youtube.com/@meanxai

Word Embedding. All rights reserved by www.youtube.com/@meanxai

MX-AI

■ Subsampling of frequent words

[Tomas Mikolov, et, al., 2013, Distributed Representations of Words and Phrases and their Compositionality]

2.3 Subsampling of Frequent Words

In very large corpora, the most frequent words can easily occur hundreds of millions of times (e.g., "in", "the", and "a"). Such words usually provide less information value than the rare words. For example, while the Skip-gram model benefits from observing the co-occurrences of "France" and "Paris", it benefits much less from observing the frequent co-occurrences of "France" and "the", as nearly every word co-occurs frequently within a sentence with "the". This idea can also be applied in the opposite direction; the vector representations of frequent words do not change significantly after training on several million examples. To counter the imbalance between the rare and frequent words, we used a simple subsampling approach: each word w_i in the training set is discarded with probability computed by the formula

$$P_{discard}(w_i) = 1 - \sqrt{\frac{t}{f(w_i)}}$$

where $f(w_i)$ is the frequency of word w_i and t is a chosen threshold, typically around 10^{-5}. We chose this subsampling formula because it aggressively subsamples words whose frequency is greater than t while preserving the ranking of the frequencies. Although this subsampling formula was chosen heuristically, we found it to work well in practice. It accelerates learning and even significantly improves the accuracy of the learned vectors of the rare words, as will be shown in the following sections.

- **The probability of keeping a word w_i**

$$P_{keep}(w_i) = \sqrt{\frac{t}{f(w_i)}}$$

$$P'_{keep}(w_i) = \left(\sqrt{\frac{f(w_i)}{t}} + 1\right)\cdot\frac{t}{f(w_i)}$$

$t = 10^{-3}$ or 10^{-5}

$f(w_i)$ is the frequency of word w_i in the corpus, often expressed as the fraction of occurrences of w_i relative to the total number of words.

https://github.com/tmikolov/word2vec/blob/master/word2vec.c

```
// The subsampling randomly discards frequent words while keeping
// the ranking same
if (sample > 0) {
   real ran = (sqrt(vocab[word].cn / (sample * train_words)) + 1)
              * (sample * train_words) / vocab[word].cn;
   next_random = next_random * (unsigned long long)25214903917 + 11;
   if (ran < (next_random & 0xFFFF) / (real)65536) continue;
}
```

example:

The frequency of 'word' = 2,000
The total number of words = 1,000,000

$$f('word') = \frac{2000}{1000000} = 0.002$$

$$P_{keep}('word') = \sqrt{\frac{0.001}{0.002}} = 0.71$$

$$P_{keep}('word') = \left(\sqrt{\frac{0.002}{0.001}} + 1\right)\cdot\frac{0.001}{0.002} = 1.21$$

MX-AI

■ Preprocessing of the dataset: Gutenberg Corpus

```python
# [MXNLP-1-04] 5.dataset.py
# Preprocess the Gutenberg Corpus and store the results for later use.
import numpy as np
import nltk
from collections import Counter
from tensorflow.keras.preprocessing.text import Tokenizer

# Download the Gutenberg corpus using NLTK.
nltk.download('punkt'); nltk.download('gutenberg'); nltk.download('stopwords')

# Define the stop words
stopwords = nltk.corpus.stopwords.words('english')
stopwords.extend(("''", "``", "--", "'s", "n't"))

# List of 18 files in the Gutenberg Corpus.
files = nltk.corpus.gutenberg.fileids()

# Count word frequencies within the 18 text files
word_freq = Counter()
list_sentences1 = []
for i, file in enumerate(files):
    text = nltk.corpus.gutenberg.raw(file)
    sentences = nltk.sent_tokenize(text.lower())

    for sentence in sentences:
        words = nltk.word_tokenize(sentence)
        words = [w for w in words if w not in stopwords if len(w) > 1]
        if len(words) >= 5:
            word_freq += Counter(words)
            list_sentences1.append(words)
    print('{}: {} -----> done.'.format(i+1, file))
sum_freq = word_freq.total()  # the sum of word frequencies
```

```
stopwords:
['a', 'about', 'above', 'after', 'again', 'against', 'ain',
 'all', 'am', 'an', 'and', 'any', 'are', 'aren', "aren't",
 'as', 'at', 'be', 'because', 'been', 'before', 'being', ...]


files:
['austen-emma.txt', 'austen-persuasion.txt',
 'austen-sense.txt', 'bible-kjv.txt', 'blake-poems.txt',
 'bryant-stories.txt', 'burgess-busterbrown.txt', ...]

word_freq.most_common(10):
[('shall', 11323), ('unto', 8897), ('said', 8289),
 ('lord', 8174), ('thou', 6551), ('one', 5969),
 ('thy', 5530), ('man', 5235), ('god', 5059), ... ]

list_sentences1[2]:
['mother', 'died', 'long', 'ago', 'indistinct',
 'remembrance', 'caresses', 'place', 'supplied',
 'excellent', 'woman', 'governess', 'fallen', 'little',
 'short', 'mother', 'affection']

The number of unique words: len(word_freq) = 51,488

The number of total sentences: len(list_sentences1) = 70,307

The sum of word frequencies: word_freq.total() = 996,967
```

Word Embedding. All rights reserved by www.youtube.com/@meanxai

MX-AI

■ Preprocessing of the dataset: Gutenberg Corpus

$$P'_{keep}(w_i) = \left(\sqrt{\frac{f(w_i)}{t}} + 1\right) \cdot \frac{t}{f(w_i)}$$

```
# the probability of keeping the word
def sampling_rate(word):
    f = word_freq[word] / sum_freq
    return (np.sqrt(f / 1e-3) + 1) * 1e-3 / f

# Subsampling of frequent words
list_sentences2 = []
for sentence in list_sentences1:
    words = [word for word in sentence \
             if np.random.rand() < sampling_rate(word)]
    if len(words) >= 5:
        list_sentences2.append(words)

# Tokenize the collection of the sentences
tokenizer = Tokenizer()
tokenizer.fit_on_texts(list_sentences2)

# Create a vocabulary
word2idx = tokenizer.word_index
word2idx['<PAD>'] = 0
idx2word = {v:k for k, v in word2idx.items()}

# Convert the words in the sentences into their corresponding
# indices in the vocabulary.
sequences = tokenizer.texts_to_sequences(list_sentences2)

# Store the vocabulary and the sequences for later use
import pickle
with open('data/gutenberg.pkl', 'wb') as f:
      pickle.dump([word2idx, idx2word, sequences], f)
```

```
list_sentences1[2]:
['mother', 'died', 'long', 'ago', 'indistinct',
 'remembrance', 'caresses', 'place', 'supplied',
 'excellent', 'woman', 'governess', 'fallen', 'little',
 'short', 'mother', 'affection']

list_sentences2[2]:
['mother', 'died', 'long', 'ago', 'indistinct',
 'remembrance', 'caresses', 'place', 'supplied',
 'excellent', 'woman', 'governess', 'fallen', 'short',
 'mother', 'affection']

The number of total sentences 69184
```

word2idx:

```
{'shall': 1,
 'unto': 2,
 'said': 3,
 'lord': 4,
 'thou': 5,
 'one': 6,
 'thy': 7,
 'man': 8,
 'god': 9,
 'thee': 10,
 ... }
```

idx2word:

```
{1: 'shall',
 2: 'unto',
 3: 'said',
 4: 'lord',
 5: 'thou',
 6: 'one',
 7: 'thy',
 8: 'man',
 9: 'god',
 10: 'thee',
 ... }
```

sequences[2]:

```
[119, 531, 67, 833, 16519,
 1702, 13259, 76, 3967,
 1040, 150, 8269, 772, 391,
 119, 996]

word2idx['mother'] = 119
word2idx['died'] = 531
...
```

Word Embedding. All rights reserved by www.youtube.com/@meanxai

MX-AI

■ Implementing a simple CBOW model

```python
# [MXNLP-1-04] 6.CBOW.py
# Implementation of a CBOW model
import numpy as np
import nltk
import tensorflow as tf
from tensorflow.keras.layers import Input, Layer, Dense
from tensorflow.keras.models import Model
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
import pickle

# Load the preprocessed Gutenberg Corpus
with open('data/gutenberg.pkl', 'rb') as f:
    word2idx, idx2word, sequences = pickle.load(f)

# Create a training dataset with 5-grams
n_grams = 5
x_train = []
y_train = []
for sequence in sequences:
    if len(sequence) >= n_grams:
        for a, b, c, d, e in nltk.ngrams(sequence, n_grams):
            x_train.append([a, b, d, e])
            y_train.append(c)

x_train = np.array(x_train)
y_train = np.array(y_train).reshape(-1, 1)

n_vocab = len(word2idx)   # vocabulary size
n_emb = 32                # word embedding vector size
```

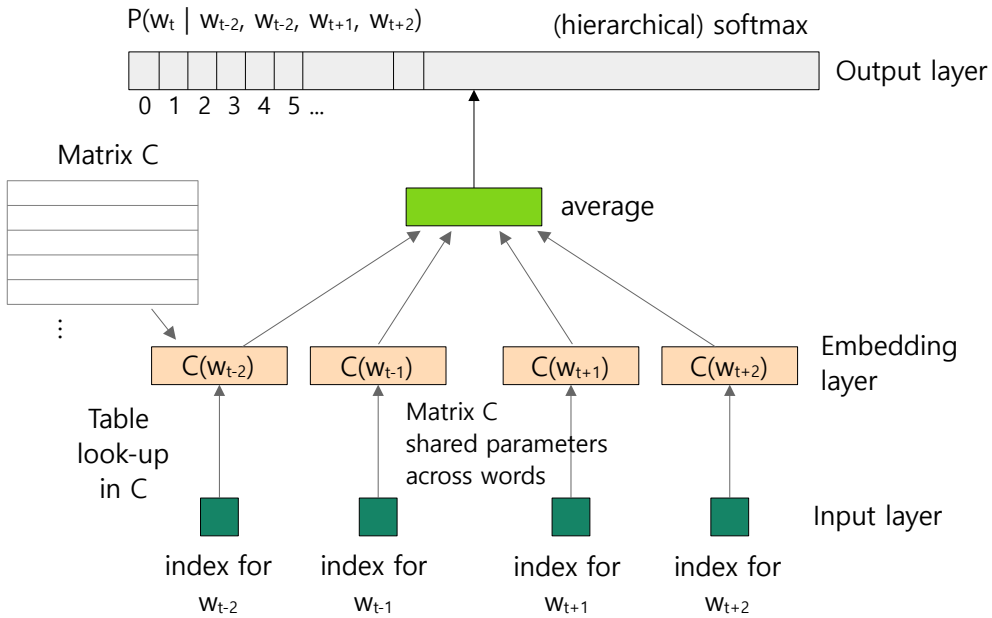
```
sequences[0]:
[174, 538, 18837, 29376, 4277, 744, 174, 512, 1273, 2446, 667,
 1512, 216, 281, 2045, 120, 7160, 266, 4927, 3152, 607, 1139,
 29377, 131, 98, 20, 1425, 5084]

The number of total sentences: len(sequences) = 69,197

word2idx:                idx2word:
{'shall': 1,             {1: 'shall',
 'unto': 2,               2: 'unto',
 'said': 3,               3: 'said',
 'lord': 4,               4: 'lord',
 'thou': 5,               5: 'thou',
 'one': 6,                6: 'one',
 'thy': 7,                7: 'thy',
 'man': 8,                8: 'man',
 'god': 9,                9: 'god',
 'thee': 10,              10: 'thee',
 ... }                    ... }

The number of unique words: len(word2idx) = 51,420

x_train[:5]:                                  y_train[:5]:
array([[  174,   538, 29376,  4277],          array([[18837],
       [  538, 18837,  4277,   744],                 [29376],
       [18837, 29376,   744,   174],                 [ 4277],
       [29376,  4277,   174,   512],                 [  744],
       [ 4277,   744,   512,  1273]])                [  174]])
```

idx2word[174] = 'emma'
idx2word[538] = 'jame'
…
idx2word[1425] = 'distress'
idx2word[5084] = 'vex'

MX-AI

■ Implementing a simple CBOW model

```python
# Custom Embedding layer
class WordEmbedding(Layer):
    def __init__(self, input_dim, emb_size, name=None):
        super(WordEmbedding, self).__init__(name = name)

        # Matrix C
        C_init = tf.random_normal_initializer()
        self.C = tf.Variable(
            initial_value = C_init(shape=(input_dim, emb_size)),
            trainable = True,
        )

    def call(self, x):
        # Table look-up in C
        selected_rows = tf.gather(self.C, x)  # (None, 4, 32)
        return selected_rows

# Create a CBOW model
x_input = Input(batch_shape=(None, x_train.shape[1]), dtype='int32')
x_emb = WordEmbedding(n_vocab, n_emb, name='emb')(x_input)
x_avg = tf.reduce_mean(x_emb, axis=1)
y_output = Dense(n_vocab, activation='softmax')(x_avg)

model = Model(x_input, y_output)
model.compile(loss = 'sparse_categorical_crossentropy',
              optimizer='adam')

# Fit the model to the training dataset
hist = model.fit(x_train, y_train, batch_size=20480,
                 shuffle = True, epochs=300)
```




```
model.summary()
 Layer (type)                Output Shape              Param #
=================================================================
 input_1 (InputLayer)       [(None, 4)]               0
 emb (WordEmbedding)         (None, 4, 32)            1645440
 tf.math.reduce_mean         (None, 32)               0
 dense (Dense)               (None, 51420)            1696860
=================================================================
Total params: 3,342,300
Trainable params: 3,342,300
Non-trainable params: 0
```

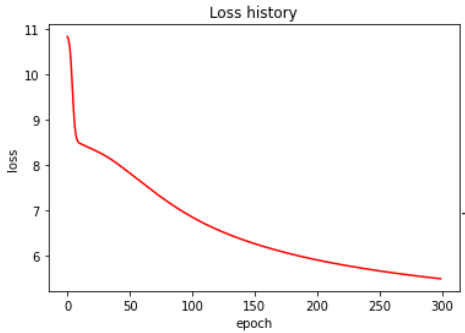
Word Embedding. All rights reserved by www.youtube.com/@meanxai

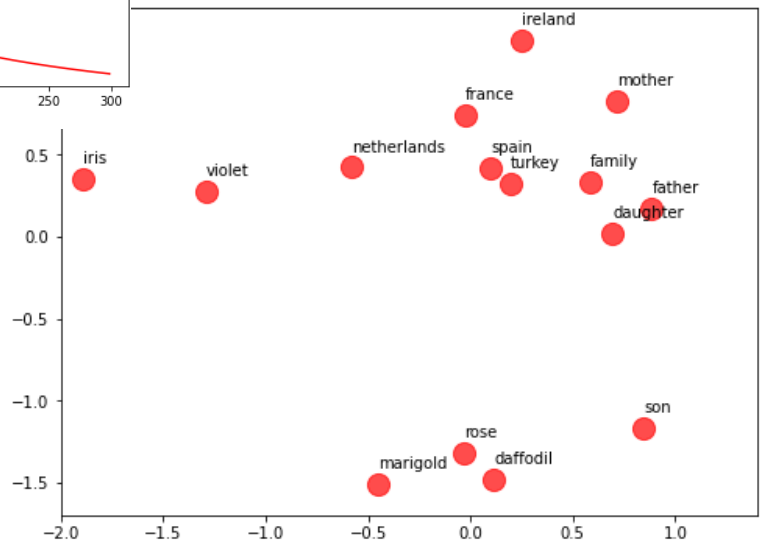
MX-AI

■ Implementing a simple CBOW model

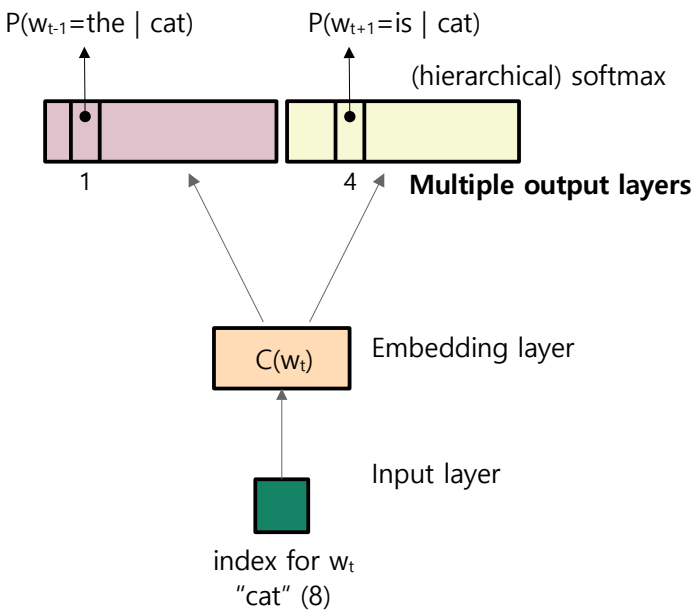
```
# Loss history
plt.plot(hist.history['loss'], color='red')
plt.title("Loss history")
plt.xlabel("epoch")
plt.ylabel("loss")
plt.show()

# Word embedding matrix C
C = model.get_layer('emb').get_weights()[0]

# To represent word embedding vectors in a two-dimensional
# vector space, we use pca to transform the 32-dimensional
# matrix C into a two-dimensional matrix.
pca = PCA(n_components = 2)
pca_C = pca.fit_transform(C)

# we represent the two-dimensional embedding vectors of
# the following words in a two-dimensional space.
words = ['daffodil', 'marigold', 'iris', 'rose', 'violet',
         'france', 'netherlands', 'spain', 'turkey', 'ireland',
         'mother', 'father', 'son', 'daughter', 'family']
words_idx = np.array([word2idx[x] for x in words])
words_vec = pca_C[words_idx]

# Let's see if words with similar meanings are close together
# in the space.
x = words_vec[:, 0]; y = words_vec[:, 1]
plt.figure(figsize=(8,6))
plt.scatter(x, y, s=200, c='red', alpha=0.7)
for i in range(len(words)):
    plt.annotate(words[i], (x[i], y[i] + 0.1))
plt.show()
```

```
C[word2idx['mother']].round(3):
[-0.301, -1.395, -0.538, -1.456, -0.   ,  0.689, -0.004,  0.784,
 -0.438, -0.031, -1.06 ,  0.691,  1.978,  0.011, -0.463, -1.363,
  0.197,  1.061,  0.681,  1.734, -0.16 , -0.675, -0.214, -0.57 ,
 -0.176,  0.487,  0.063,  0.476,  1.304,  0.383,  0.337,  1.043]
```





Word Embedding. All rights reserved by www.youtube.com/@meanxai



1-5. Word2Vec - [3]

Implementing Skip-Gram models

This video was produced in Korean and translated into English, and the voice was generated using TTS.

www.youtube.com/@meanxai

Word Embedding. All rights reserved by www.youtube.com/@meanxai

MX-AI

■ Skip-Gram models in two ways

Corpus:

```
the cat is walking in the bedroom
a dog was running in a room
the cat is running in a room
a dog is walking in a bedroom
the dog was walking in the room
```

Vocabulary:

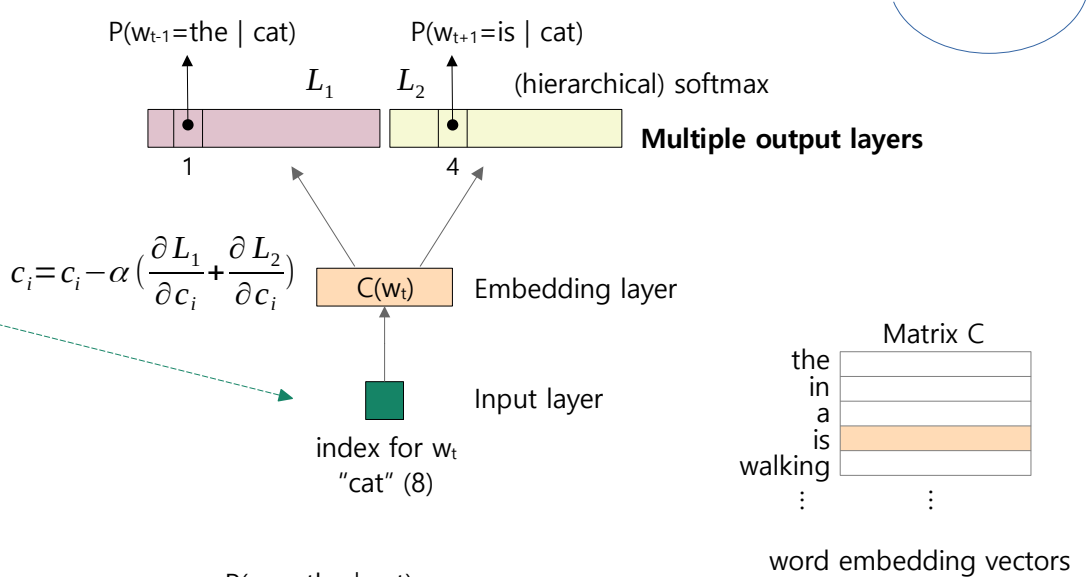
```
{'the': 1,
 'in': 2,
 'a': 3,
 'is': 4,
 'walking': 5,
 'dog': 6,
 'room': 7,
 'cat': 8,
 'bedroom': 9,
 'was': 10,
 'running': 11}
```

Training data:

w_t	w_{t-1}	w_{t+1}
cat	the	is
is	cat	walking
walking	is	in
in	walking	the
the	in	bedroom
⋮		⋮

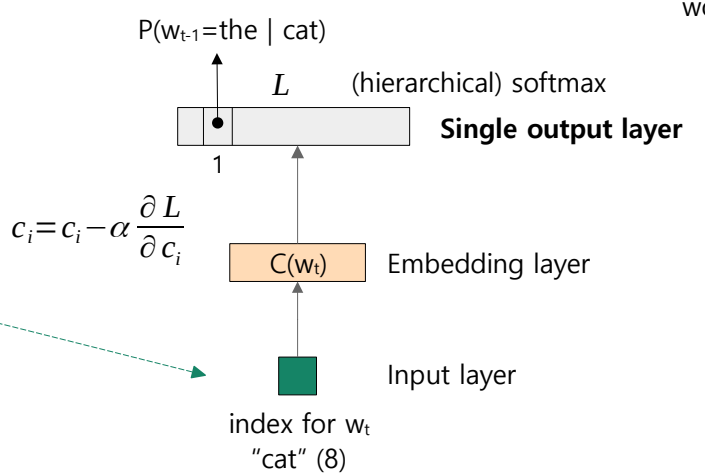
Training data:

(target) input	(context) output	
8	1	4
4	8	5
5	4	2
2	5	1
1	2	9
⋮	⋮	



Training data:

(target) input	(context) output
8	1
8	4
4	8
4	5
5	4
5	2
2	5
2	1
1	2
1	9
⋮	⋮



MX-AI

- Implementing a simple Skip-Gram model with two output layers

```python
# [MXNLP-1-05] 7.SkipGram(1).py
# Implementation of a Skip-gram model using two output layers
import numpy as np
import nltk
from tensorflow.keras.layers import Input, Embedding
from tensorflow.keras.layers import Dense, Reshape
from tensorflow.keras.models import Model
import matplotlib.pyplot as plt
import pickle

# Load the preprocessed Gutenberg Corpus
with open('data/gutenberg.pkl', 'rb') as f:
    word2idx, idx2word, sequences = pickle.load(f)

# Generating a training dataset
n_grams = 3
x_train = []
y_train = []
for sequence in sequences:
    if len(sequence) >= n_grams:
        for a, b, c in nltk.ngrams(sequence, n_grams):
            x_train.append(b)
            y_train.append([a, c])

x_train = np.array(x_train).reshape(-1, 1)
y_train = np.array(y_train)

n_vocab = len(word2idx)   # vocabulary size
n_emb = 16                # word embedding vector size
```

```
sequences[0]:
[174, 538, 18837, 29376, 4277, 744, 174, 512, 1273, 2446, 667,
 1512, 216, 281, 2045, 120, 7160, 266, 4927, 3152, 607, 1139,
 29377, 131, 98, 20, 1425, 5084]

The number of total sentences: len(sequences) = 69,197

word2idx:                idx2word:
{'shall': 1,             {1: 'shall',
 'unto': 2,               2: 'unto',
 'said': 3,               3: 'said',
 'lord': 4,               4: 'lord',
 'thou': 5,               5: 'thou',
 'one': 6,                6: 'one',
 'thy': 7,                7: 'thy',
 'man': 8,                8: 'man',
 'god': 9,                9: 'god',
 'thee': 10,              10: 'thee',
 ... }                    ... }

The number of unique words: len(word2idx) = 51,420

x_train[:5]:                  y_train[:5]:
array([[  538],               array([[  174, 18837],
       [18837],                       [  538, 29376],
       [29376],                       [18837,  4277],
       [ 4277],                       [29376,   744],
       [  744]])                      [ 4277,   174]])
```

idx2word[174] = 'emma'
idx2word[538] = 'jame'
...
idx2word[1425] = 'distress'
idx2word[5084] = 'vex'

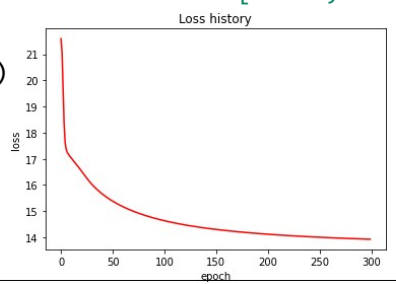
MX-AI

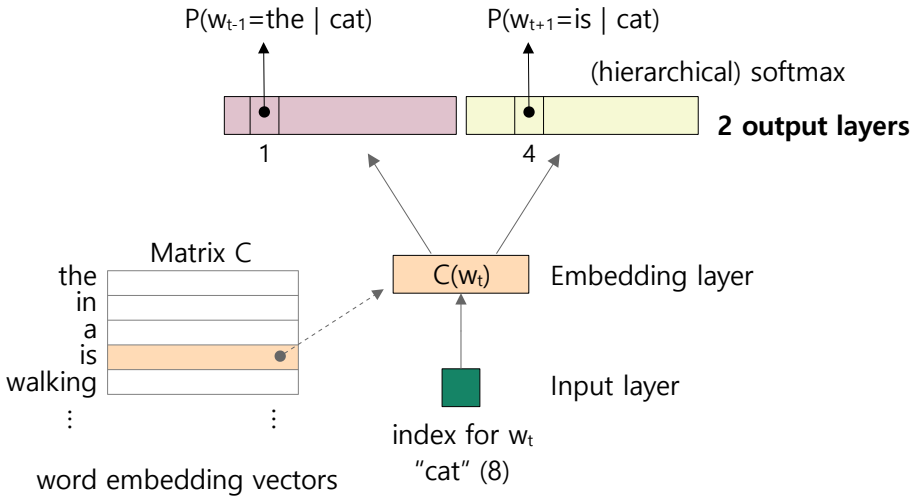
- Implementing a simple Skip-Gram model with two output layers

```
# Creating a simple Skip-Gram model
x_input = Input(batch_shape=(None, 1))
x_emb = Embedding(n_vocab, n_emb, name='emb')(x_input)  # (None, 1, 16)
x_emb = Reshape((n_emb,))(x_emb)                        # (None, 16)
y1_out = Dense(n_vocab, activation='softmax')(x_emb)    # (None, 51420)
y2_out = Dense(n_vocab, activation='softmax')(x_emb)    # (None, 51420)

model = Model(x_input, [y1_out, y2_out])
model.compile(loss = 'sparse_categorical_crossentropy',
              optimizer='adam')
model.summary()

# Fit the model to the training dataset
hist = model.fit(x_train,
                 [y_train[:, 0], y_train[:, 1]],
                 batch_size=10000,
                 epochs=300)

# Loss history
plt.plot(hist.history['loss'], color='red')
plt.title("Loss history")
plt.xlabel("epoch")
plt.ylabel("loss")
plt.show()
```

```
x_train[:5]:
array([[  538],
       [18837],
       [29376],
       [ 4277],
       [  744]])
```

```
y_train[:5]:
array([[  174, 18837],
       [  538, 29376],
       [18837,  4277],
       [29376,   744],
       [ 4277,   174]])
```





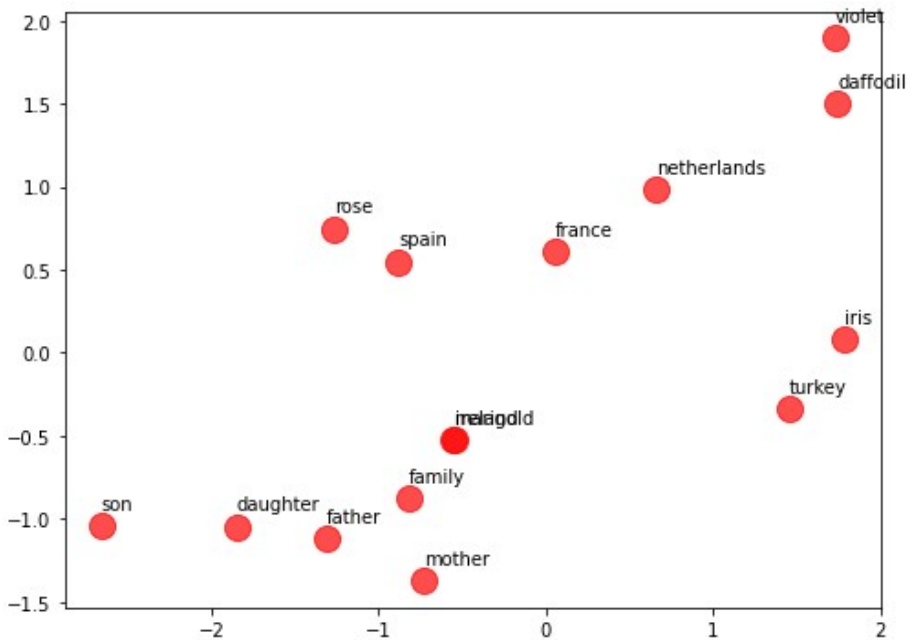

```
model.summary()
 Layer (type)                    Output Shape         Param #
=================================================================
input_1 (InputLayer)           [(None, 1)]          0
emb (Embedding)                (None, 1, 16)        822720
reshape (Reshape)              (None, 16)           0
dense (Dense)                  (None, 51420)        874140
dense_1 (Dense)                (None, 51420)        874140
=================================================================
Total params: 2,571,000
Trainable params: 2,571,000
Non-trainable params: 0
```

MX-AI

- Implementing a simple Skip-Gram model with two output layers

```
# Word embedding matrix C
C = model.get_layer('emb').get_weights()[0]

# To represent word embedding vectors in a two-dimensional
# vector space, we use PCA to transform the 32-dimensional matrix C
# into a two-dimensional matrix.
from sklearn.decomposition import PCA
pca = PCA(n_components = 2)
pca_C = pca.fit_transform(C)

# we represent the two-dimensional embedding vectors of
# the following words in a two-dimensional space.
words = ['daffodil', 'marigold', 'iris', 'rose', 'violet',
         'france', 'netherlands', 'spain', 'turkey', 'ireland',
         'mother', 'father', 'son', 'daughter', 'family']
words_idx = np.array([word2idx[x] for x in words])
words_vec = pca_C[words_idx]

# Let's see if words with similar meanings are close together
# in the space.
x = words_vec[:, 0]
y = words_vec[:, 1]
plt.figure(figsize=(8,6))
plt.scatter(x, y, s=200, c='red', alpha=0.7)
for i in range(len(words)):
    plt.annotate(words[i], (x[i], y[i] + 0.1))
plt.show()
```

```
C[word2idx['mother']].round(3):
[-0.894, -1.227, -0.228,  0.027, -0.753, -0.599,  1.198,  0.872,
  0.568, -1.875,  0.008,  0.275,  1.226,  0.898, -1.065, -0.09 ]
```



MX-AI

- Implementing a simple Skip-Gram model with a single output layer

```python
# [MXNLP-1-05] 8.SkipGram(2).py
# Implementation of a Skip-Gram model using a single output layer
import numpy as np
import nltk
from tensorflow.keras.layers import Input, Embedding
from tensorflow.keras.layers import Dense, Reshape
from tensorflow.keras.models import Model
import matplotlib.pyplot as plt
import pickle

# Load the preprocessed Gutenberg Corpus
with open('data/gutenberg.pkl', 'rb') as f:
    word2idx, idx2word, sequences = pickle.load(f)

# Create a training dataset
n_grams = 3
x_train = []
y_train = []
for sequence in sequences:
    if len(sequence) >= n_grams:
        for a, b, c in nltk.ngrams(sequence, n_grams):
            x_train.append(b)
            y_train.append(a)

            x_train.append(b)
            y_train.append(c)

x_train = np.array(x_train).reshape(-1, 1)
y_train = np.array(y_train).reshape(-1, 1)
```

```
sequences[0]:
[174, 538, 18837, 29376, 4277, 744, 174, 512, 1273, 2446, 667,
 1512, 216, 281, 2045, 120, 7160, 266, 4927, 3152, 607, 1139,
 29377, 131, 98, 20, 1425, 5084]

The number of total sentences: len(sequences) = 69,197

word2idx:                idx2word:
{'shall': 1,             {1: 'shall',
 'unto': 2,               2: 'unto',
 'said': 3,               3: 'said',
 'lord': 4,               4: 'lord',
 'thou': 5,               5: 'thou',
 'one': 6,                6: 'one',
 'thy': 7,                7: 'thy',
 'man': 8,                8: 'man',
 'god': 9,                9: 'god',
 'thee': 10,              10: 'thee',
 ... }                    ... }

The number of unique words: len(word2idx) = 51,420

x_train[:6]:                    y_train[:6]:
array([[  538],                 array([[  174],
       [  538],                        [18837],
       [18837],                        [  538],
       [18837],                        [29376],
       [29376],                        [18837],
       [29376]])                       [ 4277]])
```

idx2word[174] = 'emma'
idx2word[538] = 'jame'
…
idx2word[1425] = 'distress'
idx2word[5084] = 'vex'

Word Embedding. All rights reserved by www.youtube.com/@meanxai

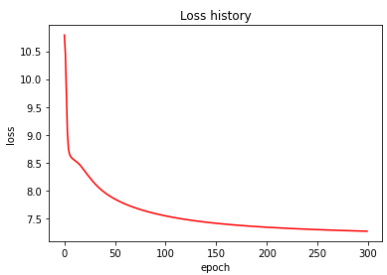
MX-AI

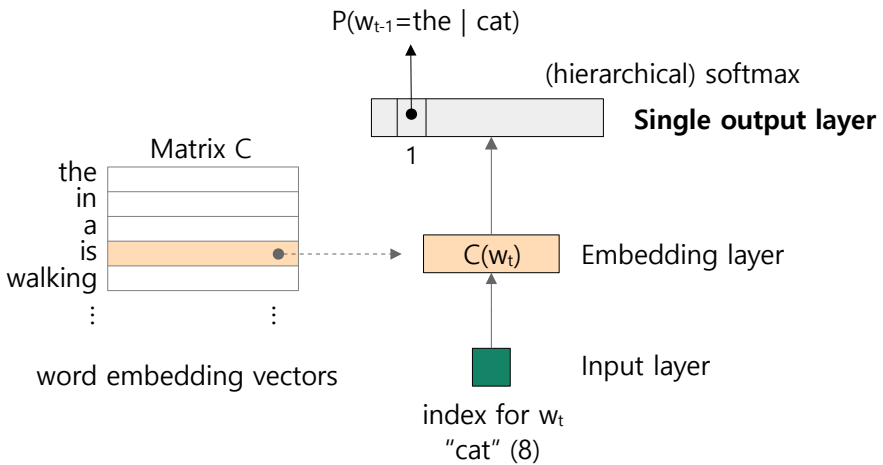
■ Implementing a simple Skip-Gram model with a single output layer

```
n_vocab = len(word2idx)   # vocabulary size
n_emb = 16                # word embedding vector size

# Creating a simple Skip-Gram model
x_input = Input(batch_shape=(None, 1))
x_emb = Embedding(n_vocab, n_emb, name='emb')(x_input)  # (None, 1, 16)
x_emb = Reshape((n_emb,))(x_emb)                        # (None, 16)
y_output = Dense(n_vocab, activation='softmax')(x_emb)  # (None, 51420)

model = Model(x_input, y_output)
model.compile(loss = 'sparse_categorical_crossentropy',
optimizer='adam')
model.summary()

# Fit the model to the training dataset
hist = model.fit(x_train, y_train,
                 batch_size=20000,
                 epochs=300)

# Loss history
plt.plot(hist.history['loss'], color='red')
plt.title("Loss history")
plt.xlabel("epoch")
plt.ylabel("loss")
plt.show()
```

```
x_train[:6]:
array([[  538],
       [  538],
       [18837],
       [18837],
       [29376],
       [29376]])
```

```
y_train[:6]:
array([[  174],
       [18837],
       [  538],
       [29376],
       [18837],
       [ 4277]])
```






```
model.summary()
 Layer (type)                Output Shape              Param #
=================================================================
 input_2 (InputLayer)       [(None, 1)]               0
 emb (Embedding)             (None, 1, 16)            822720
 reshape (Reshape)           (None, 16)               0
 dense (Dense)               (None, 51420)            874140
=================================================================
Total params: 1,696,860
Trainable params: 1,696,860
Non-trainable params: 0
```

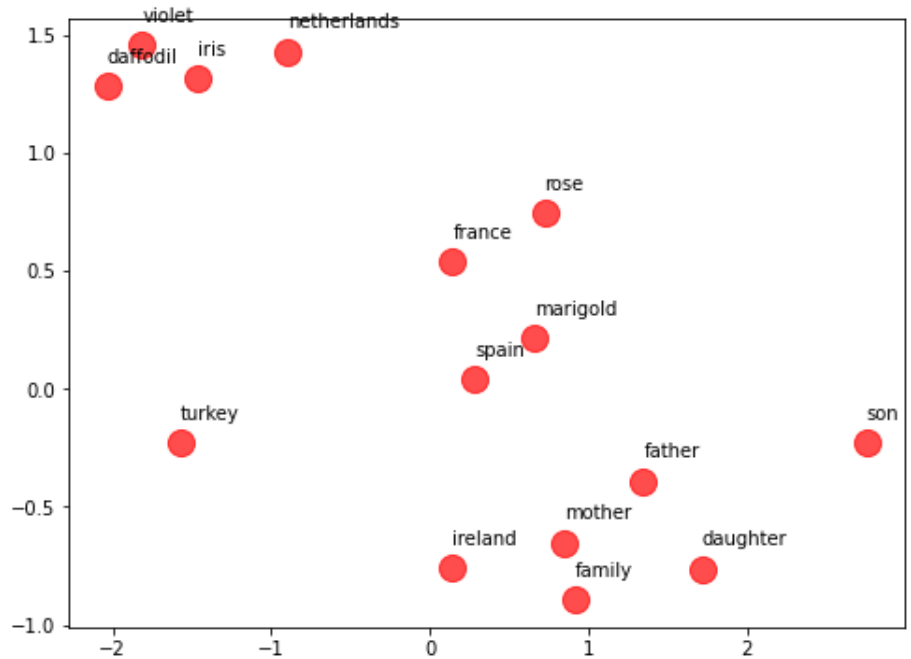
Word Embedding. All rights reserved by www.youtube.com/@meanxai

MX-AI

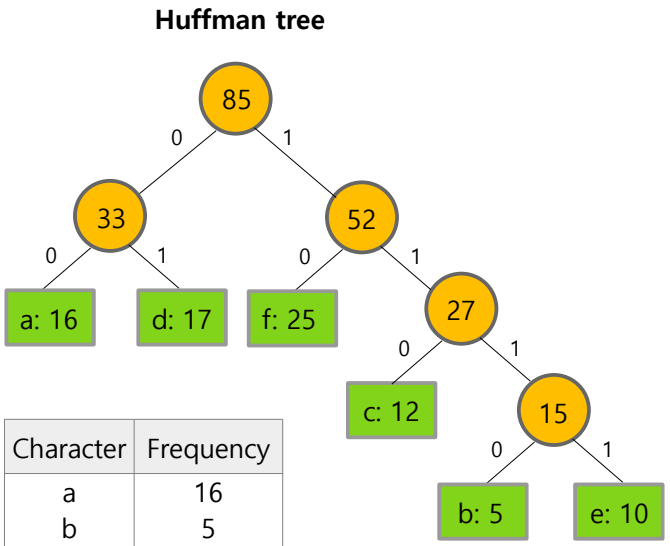
- Implementing a simple Skip-Gram model with a single output layer

```
# Word embedding matrix C
C = model.get_layer('emb').get_weights()[0]

# To represent word embedding vectors in a two-dimensional
# vector space, we use pca to transform the 32-dimensional matrix C
# into a two-dimensional matrix.
from sklearn.decomposition import PCA
pca = PCA(n_components = 2)
pca_C = pca.fit_transform(C)

# we represent the two-dimensional embedding vectors of
# the following words in a two-dimensional space.
words = ['daffodil', 'marigold', 'iris', 'rose', 'violet',
         'france', 'netherlands', 'spain', 'turkey', 'ireland',
         'mother', 'father', 'son', 'daughter', 'family']
words_idx = np.array([word2idx[x] for x in words])
words_vec = pca_C[words_idx]

# Let's see if words with similar meanings are close together
# in the space.
x = words_vec[:, 0]
y = words_vec[:, 1]
plt.figure(figsize=(8,6))
plt.scatter(x, y, s=200, c='red', alpha=0.7)
for i in range(len(words)):
    plt.annotate(words[i], (x[i], y[i] + 0.1))
plt.show()
```

```
C[word2idx['mother']].round(3):
[ 0.633,  0.089, -0.402, -0.861,  0.119, -1.128,  0.622,  0.302,
  1.109, -0.267,  0.411, -0.142,  0.553,  0.388,  0.795, -0.566]
```



Word Embedding. All rights reserved by www.youtube.com/@meanxai



Character	Frequency
a	16
b	5
c	12
d	17
e	10
f	25

1-6. Word2Vec - [4]

Huffman Tree

This video was produced in Korean and translated into English, and the voice was generated using TTS.

www.youtube.com/@meanxai

Word Embedding. All rights reserved by www.youtube.com/@meanxai

MX-AI

■ Huffman code

[Foundations of algorithm 5th edition by Richard E. Neapolitan in 2015]

4.4 Huffman Code

Even though the capacity of secondary storage devices keeps getting larger and their cost keeps getting smaller, the devices continue to fill up due to increased storage demands. Given a data file, it would therefore be desirable to find a way to store the file as efficiently as possible. The problem of data compression is to find an efficient method for encoding a data file. Next, we discuss the encoding method, called Huffman code, and a greedy algorithm for finding a Huffman encoding for a given file.

A common way to represent a file is to use a binary code. In such a code, each character is represented by a unique binary string, called the codeword. A fixed length binary code represents each character using the same number of bits. For example, suppose our character set is {a, b, c}. Then we could use 2 bits to code each character, since this gives us four possible codewords and we need only three.

We could code as follows:

a: 00 b: 01 c: 11

Given this code, if our file is

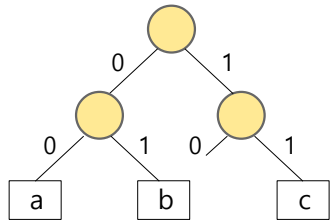
ababcbbc (4.1)

our encoding is

000100011101010111

...

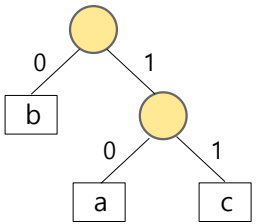
▪ **Fixed-length binary code**



a file:

ababcbbc $\xrightarrow[\text{encoding}]{\text{a: 00, b: 01, c: 11}}$ 000100011101010111

▪ **Variable-length binary code**



ababcbbc $\xrightarrow[\text{encoding}]{\text{a: 10, b: 0, c: 11}}$ 1001001100011

Word Embedding. All rights reserved by www.youtube.com/@meanxai

MX-AI

■ Huffman code: Example

Table 4.1 Three codes for the same file. C3 is optimal.

Character	Frequency	C1 (Fixed-length)	C2	C3 (Huffman)
a	16	000	10	00
b	5	001	11110	1110
c	12	010	1110	110
d	17	011	110	01
e	10	100	11111	1111
f	25	101	0	10

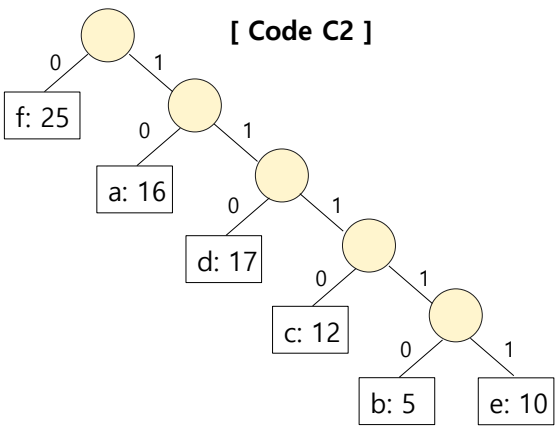
The number of bits for each encoding:

Bits(C1) = 16 x 3 + 5 x 3 + 12 x 3 + 17 x 3 + 10 x 3 + 25 x 3 = 255

Bits(C2) = 16 x 2 + 5 x 5 + 12 x 4 + 17 x 3 + 10 x 5 + 25 x 1 = 231

Bits(C3) = 16 x 2 + 5 x 4 + 12 x 3 + 17 x 2 + 10 x 4 + 25 x 2 = 212

$$Bits(T) = \sum_{i=1}^{n} frequency(v_i) \cdot depth(v_i)$$



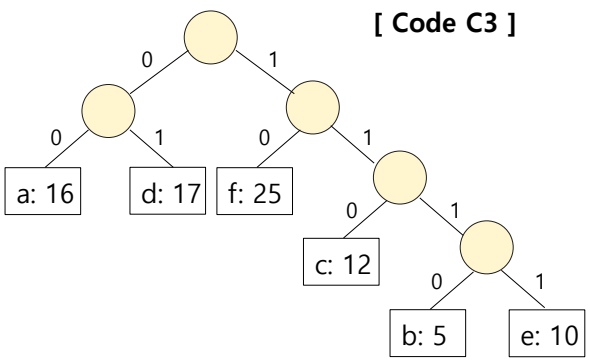


Figure 4.10 The binary character code for Code C2 in Example 4.7 appears in (a), while the one for Code C3 (Huffman) appears in (b).

Word Embedding. All rights reserved by www.youtube.com/@meanxai

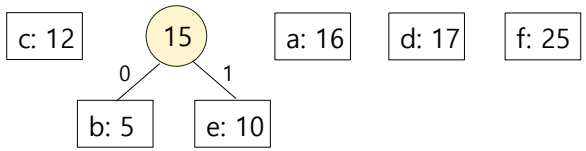
MX-AI

- Huffman algorithm: Creating a Huffman tree

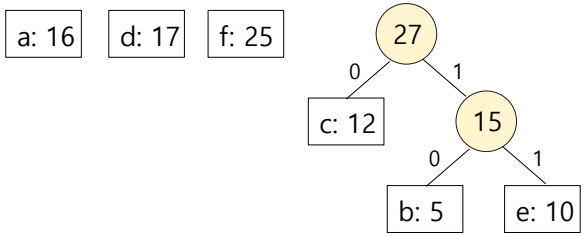
Step-1: a: 16 | b: 5 | c: 12 | d: 17 | e: 10 | f: 25

Step-2: b: 5 | e: 10 | c: 12 | a: 16 | d: 17 | f: 25 — Priority queue (PQ)

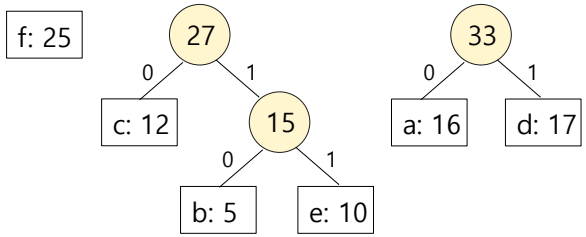
Step-3:



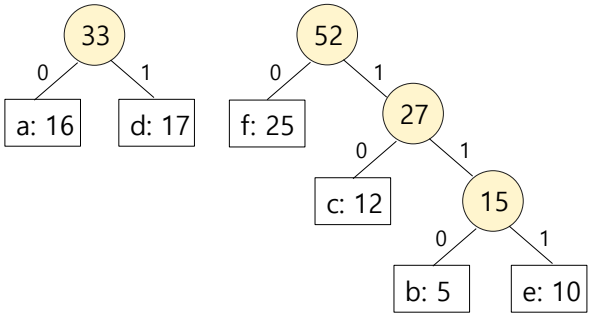
Step-4:



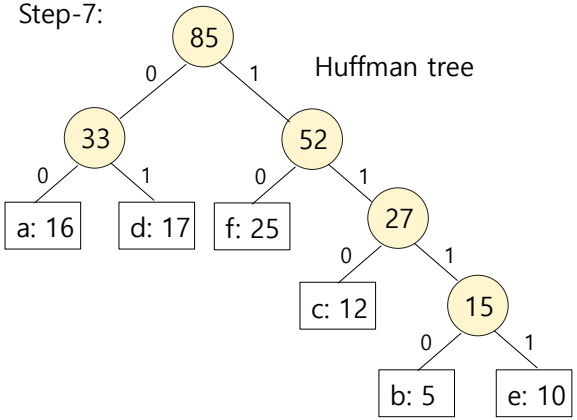
Step-5:



Step-6:



Step-7:



Huffman tree

```
struct nodetype {
   char symbol;
   int frequency;
   nodetype* left;
   nodetype* right;
}

for (i=1; i <= n-1; i++) {
   remove(PQ, p);
   remove(PQ, q);
   r = new nodetype;
   r->left = p;
   r->right = q;
   r->frequency = p->frequency +
                  q->frequency;
   insert(PQ, r);
}
```

a: 00	d: 01
b: 1110	e: 1111
c: 110	f: 10

A compressed bitstream:

0111010...

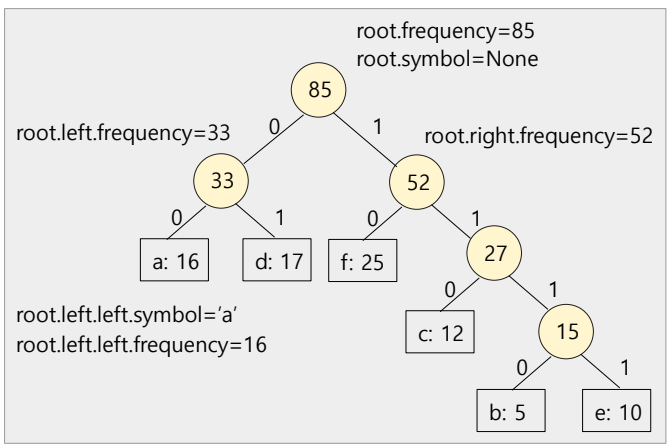
Word Embedding. All rights reserved by www.youtube.com/@meanxai

MX-AI

■ Creating a Huffman tree

```python
# [MXNLP-1-06] 9.HuffmanTree.py : Creating a Huffman tree
import numpy as np
from queue import PriorityQueue

class nodetype:
    def __init__(self, symbol, freq):
        self.symbol = symbol    # the value of a character
        self.frequency = freq   # the number of times the
                                # character is in the file
        self.left = None
        self.right = None

    def __lt__(self, other):
        return self.frequency < other.frequency

# n: the number of characters in the file
# PQ: a priority queue
def huffman_tree(n, PQ):
    for _ in range(n-1):
        p = PQ.get()                    # remove(PQ, p)
        q = PQ.get()                    # remove(PQ, q)
        sum_freq = p.frequency + q.frequency
        r = nodetype(None, sum_freq) # new nodetype
        r.left = p
        r.right = q
        PQ.put(r)                       # insert(PQ, r)
    r = PQ.get()                        # remove(PQ, r)
    return r

chars = ['a', 'b', 'c', 'd', 'e', 'f']
freqs = [16, 5, 12, 17, 10, 25]
```

```python
# Construct a priority queue for Huffman coding
PQ = PriorityQueue()
for i in range(len(chars)):
    node = nodetype(chars[i], freqs[i])
    PQ.put(node)

# Build a Huffman tree
root = huffman_tree(len(chars), PQ)

# Given a Huffman tree, find Huffman code for each char.
def huffman_code(node, c=[], code={}):
    if node is not None:
        if node.symbol is None:  # not a leaf node
            huffman_code(node.left, c + [0], code)
            huffman_code(node.right, c + [1], code)
        else: # leaf node
            code[node.symbol] = c

    return code

huffman_code = huffman_code(root)
print(huffman_code)
```

```
{'a': [0, 0],
 'd': [0, 1],
 'f': [1, 0],
 'c': [1, 1, 0],
 'b': [1, 1, 1, 0],
 'e': [1, 1, 1, 1]}
```

```
struct nodetype {
   char symbol;
   int frequency;
   nodetype* left;
   nodetype* right;
}

for (i=1; i <= n-1; i++) {
   remove(PQ, p);
   remove(PQ, q);
   r = new nodetype;
   r->left = p;
   r->right = q;
   r->frequency = p->frequency +
                  q->frequency;
   insert(PQ, r);
}
```